

JAVA 17 OCP · YDS STUDY SERIES

Unit 02 · Operators · Eksiksiz Çift Dilli Kaynak Notu

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu düzenlenebilir ana not, kaynak PDF'deki bölümün sayfa sırasını korur. Her anlamlı English kaynak satırı hemen ardından doğal Türkçe karşılığıyla verilir; kod ve terminal komutları ise çevrilmeden yalnızca bir kez gösterilir. Ayrıntılı dil çalışması için [vocabulary](#) ve [grammar notes](#) dosyalarını kullan.

Kaynak ve kapsam özeti

- **Kaynak:** [exam_lecture/OCP_Java_SE17_Chapter1den_Itibaren.pdf](#)
- **PDF kapsamı:** 0065–0100
- **İşlenen sayfa sayısı:** 36
- **İşlenen anlamlı kaynak girdisi:** 1298
- **Çıkarılan öğeler:** Yalnızca tekrarlanan running header, footer ve sayfa numarası
- **OCR düzeltmeleri:** Soft hyphen ve satır bölünmesi kaynaklı tireler teknik yazıma getirildi
- **İzlenebilirlik:** Her kaynak PDF sayfası kaydedilir ve kapsam doğrulaması belgenin sonunda özetlenir

İçindekiler

- [Kaynak cevaplarıyla kontrol](#) · Soruları çözdükten sonra aç.
- 1 [Understanding Java Operators](#)
- 2 [Operator Precedence](#)
- 3 [Applying Unary Operators](#)
- 4 [Working with Binary Arithmetic Operators](#)
- 5 [Working with Assignment Operators](#)
- 6 [Comparing Values](#)
- 7 [Making Decisions with the Ternary Operator](#)
- 8 [Summary](#)
- 9 [Exam Essentials](#)
- 10 [Review Questions](#)

Kaynak PDF sayfası 65

Chapter

Türkçe başlık: Bölüm

English: 2

Türkçe: 2

Operators

Türkçe başlık: Operatörler

OCP EXAM OBJECTIVES COVERED IN

Türkçe başlık: OCP SINAVININ HEDEFLERİ

English: THIS CHAPTER:

Türkçe: BU BÖLÜM:

English: [x] Handling date, time, text, numeric and boolean values

Türkçe: [x] Tarih, saat, metin, sayısal ve boolean değerlerini işleyin

English: [x] Use primitives and wrapper classes including Math API, parentheses, type promotion, and casting to evaluate arithmetic and boolean expressions

Türkçe: [x] Aritmetik ve boolean ifadeleri değerlendirmek için Math API, parantezler, type promotion ve casting dahil primitive type'ları ve wrapper class'ları kullanın

Kaynak PDF sayfası 66

English: The previous chapter talked a lot about defining variables, but what can you do with a variable once it is created? This chapter introduces operators and shows how you can use them to combine existing variables and create new values. It shows you how to apply operators to various primitive data types, including introducing you to operators that can be applied to objects.

Türkçe: Önceki bölümde variable declaration üzerinde durduk. Peki bir variable oluşturulduktan sonra onunla neler yapılabilir? Bu bölüm operator'ları tanıtır; mevcut variable'ları birleştirip yeni value'lar üretmek için nasıl kullanılacaklarını gösterir. Ayrıca operator'ların çeşitli primitive type'lara ve object'lere uygulanmasını ele alır.

Understanding Java Operators

Türkçe başlık: Java Operatörlerini Anlamak

English: Before we get into the fun stuff, let's cover a bit of terminology. A Java operator is a special symbol that can be applied to a set of variables, values, or literals—referred to as operands—and that returns a result. The term operand, which we use throughout this chapter, refers to the value or variable the operator is being applied to. Figure 2.1 shows the anatomy of a Java operation.

Türkçe: Konuya başlamadan önce terminolojiyi netleştirelim. Java operator'ı, operand adı verilen variable, value veya literal'lara uygulanıp bir result üreten özel semboldür. Bu bölümde operand, operator'ın uygulandığı value veya variable anlamında kullanılır. Şekil 2.1 bir Java operation'ının yapısını gösterir.

English figure caption: FIGURE 2.1 Java operation

Türkçe şekil başlığı: ŞEKİL 2.1 Java işlemi

English figure label: Operands

Türkçe şekil etiketi: Operand'lar

```
var c = a + b;
```

English figure label: Operator

Türkçe şekil etiketi: Operator

English figure label: Result assigned to c

Türkçe şekil etiketi: Result c'ye atanır

English: The output of the operation is simply referred to as the result. Figure 2.1 actually contains a second operation, with the assignment operator (=) being used to store the result in variable c.

Türkçe: Operation'ın çıktısına result denir. Şekil 2.1 ayrıca result'ı c variable'ında saklayan assignment operator (=) ile ikinci bir operation içerir.

English: We're sure you have been using the addition (+) and subtraction (-) operators since you were a little kid. Java supports many other operators that you need to know for the exam.

Türkçe: Toplama (+) ve çıkarma (-) operatörlerini küçüklüğünüzden beri kullandığınızdan eminiz. Java, sınav için bilmeniz gereken diğer birçok operator'ı destekler.

English: While many should be review for you, some (such as the compound assignment operators) may be new to you.

Türkçe: Bunların çoğu sizin için tekrar niteliğinde olsa da compound assignment operator'ları gibi bazıları yeni olabilir.

Types of Operators

Türkçe başlık: Operatör Türleri

English: Java supports three flavors of operators: unary, binary, and ternary. These types of operators can be applied to one, two, or three operands, respectively. For the exam, you need to know

Türkçe: Java üç operator türünü destekler: unary, binary ve ternary. Bunlar sırasıyla bir, iki ve üç operand'a uygulanır. Sınav için şunları bilmelisiniz:

Kaynak PDF sayfası 67

English: a specific subset of Java operators, how to apply them, and the order in which they should be applied.

Türkçe: Belirli Java operator'larını, bunların nasıl uygulanacağını ve evaluation order'larını.

English: Java operators are not necessarily evaluated from left-to-right order. In this following example, the second expression is actually evaluated from right to left, given the specific operators involved:

Türkçe: Java operatörlerinin mutlaka soldan sağa sırayla değerlendirilmesi gerekmez. Aşağıdaki örnekte, ikinci ifade, ilgili belirli operator'lar göz önüne alındığında aslında sağdan sola doğru değerlendirilir:

Editör notu · Precedence ve evaluation order: Kaynaktaki “sağdan sola” ifadesi bu örnekte işlemlerin gruplanmasını anlatır. Operand'lar Java'da soldan sağa değerlendirilir; sağdaki yüksek öncelikli işlem, soldaki operand'ın side effect'ini öne geçerek atlayamaz. [JLS 17 §15.7](#).

```
int cookies = 4;
double reward = 3 + 2 * --cookies;
System.out.print("Zoo animal receives: "+reward+" reward points");
```

English: In this example, you first decrement cookies to 3, then multiply the resulting value by 2, and finally add 3. The value then is automatically promoted from 9 to 9.0 and assigned to reward. The final values of reward and cookies are 9.0 and 3, respectively, with the following printed:

Türkçe: Bu örnekte önce `cookies` decrement edilerek 3 yapılır, sonra result 2 ile çarpılır ve son olarak 3 eklenir. Value otomatik olarak 9'dan 9.0'a promote edilir ve `reward` variable'ına atanır. `reward` ve `cookies` variable'larının son value'ları sırasıyla 9.0 ve 3'tür; aşağıdaki çıktı oluşur:

```
Zoo animal receives: 9.0 reward points
```

English: If you didn't follow that evaluation, don't worry. By the end of this chapter, solving problems like this should be second nature.

Türkçe: Evaluation adımlarını izleyemediyseniz endişelenmeyin; bölüm sonunda bu tür soruları çözmek doğal bir beceri hâline gelecektir.

Operator Precedence

Türkçe başlık: Operatör Önceliği

English: When reading a book or a newspaper, some written languages are evaluated from left to right, while some are evaluated from right to left. In mathematics, certain operators can override other operators and be evaluated first. Determining which operators are evaluated in what order is referred to as operator precedence. In this manner, Java more closely follows the rules for mathematics. Consider the following expression:

Türkçe: Bazı yazı dilleri soldan sağa, bazıları sağdan sola okunur. Matematikte ise bazı operator'lar diğerlerinden önce uygulanır. Bu işlem sırasını operator precedence (işlem önceliği) belirler. Java da bu açıdan matematik kurallarını izler. Aşağıdaki expression'ı inceleyin:

```
var perimeter = 2 * height + 2 * length;
```

English: Let's apply some optional parentheses to demonstrate how the compiler evaluates this statement:

Türkçe: compiler'ın bu ifadeyi nasıl değerlendirdiğini göstermek için bazı isteğe bağlı parantezleri uygulayalım:

```
var perimeter = ((2 * height) + (2 * length));
```

English: The multiplication operator (*) has a higher precedence than the addition operator (+), so the height and length are both multiplied by 2 before being added together. The assignment operator (=) has the lowest order of precedence, so the assignment to the perimeter variable is performed last.

Türkçe: Multiplication operator (*), addition operator'dan (+) daha yüksek precedence'a sahiptir. Bu nedenle `height` ve `length`, toplanmadan önce ayrı ayrı 2 ile çarpılır. Assignment operator (=) en düşük precedence'a sahip olduğundan `perimeter` variable'ına assignment en son yapılır.

English: Unless overridden with parentheses, Java operators follow order of operation, listed in Table 2.1, by decreasing order of operator precedence. If two operators have the same level of precedence, then Java guarantees left-to-right evaluation for most operators other than the ones marked in the table.

Türkçe: Parantezlerle değiştirilmedikçe Java operator'ları Tablo 2.1'de yüksekten düşüğe gösterilen precedence sırasına göre gruplanır. Aynı precedence düzeyindeki operator'ların çoğu soldan sağa birleşir; ters yönde birleşenler tabloda ayrıca işaretlenmiştir.

Kaynak PDF sayfası 68

English table caption: TABLE 2.1 Order of operator precedence

Türkçe tablo başlığı: TABLO 2.1 Operator precedence sırası

Operator / Operator	Symbols and examples / Semboller ve örnekler	Evaluation order / Değerlendirme yönü
Post-unary operators / Postfix unary operator'lar	<code>expression++, expression--</code>	Left-to-right / Soldan sağa
Pre-unary operators / Prefix unary operator'lar	<code>++expression, --expression</code>	Left-to-right / Soldan sağa
Other unary operators / Diğer unary operator'lar	<code>-, !, ~, +, (type)</code>	Right-to-left / Sağdan sola
Cast / Casting	<code>(Type) reference</code>	Right-to-left / Sağdan sola
Multiplication/division/modulus / Çarpma-bölme-modulus	<code>*, /, %</code>	Left-to-right / Soldan sağa
Addition/subtraction / Toplama-çıkarma	<code>+, -</code>	Left-to-right / Soldan sağa
Shift operators	<code><<, >>, >>></code>	Left-to-right / Soldan sağa
Relational operators / İlişkisel operator'lar	<code><, >, <=, >=, instanceof</code>	Left-to-right / Soldan sağa

Operator / Operator	Symbols and examples / Semboller ve örnekler	Evaluation order / Değerlendirme yönü
Equal to/not equal to / Eşit-eşit değil	<code>==, !=</code>	Left-to-right / Soldan sağa
Logical AND	<code>&</code>	Left-to-right / Soldan sağa
Logical exclusive OR	<code>^</code>	Left-to-right / Soldan sağa
Logical inclusive OR	<code>\ </code>	Left-to-right / Soldan sağa
Conditional AND	<code>&&</code>	Left-to-right / Soldan sağa
Conditional OR	<code>\ </code>	Left-to-right / Soldan sağa
Ternary operators / Ternary operator'lar	<code>boolean expression ? expression1 : expression2</code>	Right-to-left / Sağdan sola
Assignment operators / Assignment operator'ları	<code>=, +=, -=, *=, /=, % =, &=, ^=, \ =, <<=, >>=, >>>=</code>	Right-to-left / Sağdan sola
Arrow operator	<code>-></code>	Right-to-left / Sağdan sola

English: We recommend keeping Table 2.1 handy throughout this chapter. For the exam, you need to memorize the order of precedence in this table. Note that you won't be tested on some operators, like the shift operators, although we recommend that you be aware of their existence.

Türkçe: Bu bölüm boyunca Tablo 2.1'i el altında bulundurmanızı öneririz. Sınav için bu tablodaki precedence sırasını ezberlemeniz gerekir. Shift operator'ları gibi bazı operator'lardan doğrudan sorumlu olmayacaksınız; yine de bunların varlığını bilmenizi öneririz.

Kaynak PDF sayfası 69

English: The arrow operator (`->`), sometimes called the arrow function or lambda operator, is a binary operator that represents a relationship between two operands. Although we won't cover the arrow operator in this chapter, you will see it used in switch expressions in Chapter 3, "Making Decisions," and in lambda expressions starting in Chapter 8, "Lambdas and Functional Interfaces."

Türkçe: Bazen arrow function veya lambda operator olarak da adlandırılan arrow operator (`->`), iki operand arasındaki ilişkiyi temsil eden binary bir operator'dır. Bu bölümde arrow operator'ı ele almayacağız; ancak onu Chapter 3 "Making Decisions" içindeki switch expression'larda ve Chapter 8 "Lambdas and Functional Interfaces" ile başlayan lambda expression'larda göreceksiniz.

Applying Unary Operators

Türkçe başlık: Unary Operator'ları Uygulamak

English: By definition, a unary operator is one that requires exactly one operand, or variable, to function. As shown in Table 2.2, they often perform simple tasks, such as increasing a numeric variable by one or negating a boolean value.

Türkçe: Tanım gereği unary operator tam olarak bir operand üzerinde çalışır. Tablo 2.2'de gösterildiği gibi bu operator'lar çoğunlukla numeric variable'ı bir artırmak veya boolean value'yu tersine çevirmek gibi basit işlemler yapar.

English table caption: TABLE 2.2 Unary operators

Türkçe tablo başlığı: TABLO 2.2 Unary operator'lar

Operator / Operator	Example / Örnek	Description / Açıklama
Logical complement	<code>!a</code>	Inverts a boolean's logical value. / Boolean değerini mantıksal değerini tersine çevirir.
Bitwise complement	<code>~b</code>	Inverts all 0s and 1s in a number. / Bir sayıdaki bütün 0 ve 1 bitlerini tersine çevirir.
Plus	<code>+c</code>	Indicates a number is positive, although numbers are assumed to be positive in Java unless accompanied by a negative unary operator. / Negatif unary operator eşlik etmedikçe Java'da sayılar pozitif kabul edilir; bu operator pozitifliği açıkça belirtir.
Negation or minus	<code>-d</code>	Indicates a literal number is negative or negates an expression. / Bir literal'ın negatif olduğunu belirtir veya bir expression'ın işaretini tersine çevirir.
Increment	<code>++e, f++</code>	Increments a value by 1. / Değeri 1 artırır.
Decrement	<code>--f, h--</code>	Decrements a value by 1. / Değeri 1 azaltır.
Cast	<code>(String)i</code>	Casts a value to a specific type. / Bir değeri belirli bir type'a cast eder.

English: Even though Table 2.2 includes the casting operator, we postpone discussing casting until the “Assigning Values” section later in this chapter, since that is where it is commonly used.

Türkçe: Tablo 2.2 casting operator'ını içerse de casting konusunu, yaygın olarak kullanıldığı bu bölümdeki “Assigning Values” kısmına kadar erteliyoruz.

Kaynak PDF sayfası 70

Complement and Negation Operators

Türkçe başlık: Tümlleme ve Olumsuzluk Operatörleri

English: Since we're going to be working with a lot of numeric operators in this chapter, let's get the boolean one out of the way first. The logical complement operator (!) flips the value of a boolean expression. For example, if the value is true, it will be converted to false, and vice versa. To illustrate this, compare the outputs of the following statements:

Türkçe: Bu bölümde çok sayıda numeric operator kullanacağımız için önce boolean operator'ı ele alalım. Logical complement operator (!), bir boolean expression'ın value'sunu tersine çevirir: `true`yu `false`, `false`u `true` yapar. Aşağıdaki statement'ların output'larını karşılaştırın:

```
boolean isAnimalAsleep = false;
System.out.print(isAnimalAsleep); // false
isAnimalAsleep = !isAnimalAsleep;
System.out.print(isAnimalAsleep); // true
```

English: For the exam, you also need to know about the bitwise complement operator (~), which flips all of the 0s and 1s in a number. It can only be applied to integer numeric types such as byte, short, char, int, and long. Let's try an example. For simplicity, we only show the last four bits (instead of all 32 bits).

Türkçe: Sınav için bir sayının bütün 0 ve 1 bitlerini tersine çeviren bitwise complement operator'ı (~) da bilmelisiniz. Bu operator yalnızca byte, short, char, int ve long gibi integral numeric type'lara uygulanabilir. Basitlik için örnekte 32 bitin tamamı yerine yalnızca son dört bit gösterilir.

```
int value = 3; // Stored as 0011
int complement = ~value; // Stored as 1100
System.out.println(value); // 3
System.out.println(complement); // -4
```

English: Relax! You don't need to know how to do complicated bit arithmetic on the exam, as long as you remember this rule: to find the bitwise complement of a number, multiply it by negative one and then subtract one.

Türkçe: Rahatlayın! Şu kuralı hatırladığınız sürece, sınavda karmaşık bit aritmetiği yapmayı bilmenize gerek yok: Bir sayının bit düzeyinde tümleyenini bulmak için, bunu negatif bir ile çarpın ve ardından bir çıkarın.

```
System.out.println(-1*value -1); // -4
System.out.println(-1*complement -1); // 3
```

English: Moving on to more common operators, the negation operator (-) reverses the sign of a numeric expression, as shown in these statements:

Türkçe: Daha yaygın operatörlere geçildiğinde, olumsuzluk operator'ı (-), aşağıdaki ifadelerde gösterildiği gibi sayısal bir ifadenin işaretini tersine çevirir:

```
double zooTemperature = 1.21;
System.out.println(zooTemperature); // 1.21
zooTemperature = -zooTemperature;
System.out.println(zooTemperature); // -1.21
zooTemperature = -(-zooTemperature);
System.out.println(zooTemperature); // -1.21
```

English: Notice that in the last example we used parentheses, (), for the negation operator, -, to apply the negation twice. If we had instead written --, then it would have been interpreted as the decrement operator and printed -2.21. You will see more of that decrement operator shortly.

Türkçe: Son örnekte negation operator'ını (-) iki kez uygulamak için parantez kullandığımıza dikkat edin. Bunun yerine -- yazsaydık Java bunu decrement operator olarak yorumlar ve tam olarak -2.21 yazdırırdı. Decrement operator'ını birazdan daha ayrıntılı göreceksiniz.

English: Based on the description, it might be obvious that some operators require the variable or expression they're acting on to be of a specific type. For example, you cannot apply a negation operator (-) to a boolean expression, nor can you apply a logical complement operator (!) to a numeric expression. Be wary of questions on the exam that try to do this,

Türkçe: Bazı operator'lar operand'ın belirli bir type'ta olmasını gerektirir. Örneğin boolean expression'a numeric negation operator (-) veya numeric expression'a logical complement operator (!) uygulanamaz. Sınavda bu tür type uyumsuzluklarına dikkat edin;

Kaynak PDF sayfası 71

English: as they cause the code to fail to compile. For example, none of the following lines of code will compile:

Türkçe: kodun derlenememesine neden oldukları için. Örneğin aşağıdaki kod satırlarından hiçbir derlenmeyecektir:

```
int pelican =!5; // DOES NOT COMPILE
boolean penguin = - true; // DOES NOT COMPILE
boolean peacock =!0; // DOES NOT COMPILE
```

English: The first statement will not compile because in Java you cannot perform a logical inversion of a numeric value. The second statement does not compile because you cannot numerically negate a boolean value; you need to use the logical inverse operator. Finally, the last statement does not compile because you cannot take the logical complement of a numeric value, nor can you assign an integer to a boolean variable.

Türkçe: İlk statement derlenmez; Java'da numeric value'ya logical negation uygulanamaz. İkinci statement da derlenmez; boolean value numeric olarak negate edilemez, bunun için logical complement operator kullanılmalıdır. Son statement ise numeric value'ya logical complement uygulanmaya ve **boolean** variable'a integer atanmaya çalışıldığı için derlenmez.

English: Keep an eye out for questions on the exam that use numeric values (such as 0 or 1) with boolean expressions. Unlike in some other programming languages, in Java, 1 and true are not related in any way, just as 0 and false are not related.

Türkçe: Boolean expression'larla 0 veya 1 gibi numeric value'ları bir arada kullanan sınav sorularına dikkat edin. Bazı dillerin aksine Java'da 0 ile **false** veya 1 ile **true** arasında hiçbir eşdeğerlik yoktur.

Increment and Decrement Operators

Türkçe başlık: Arttırma ve Azaltma Operatörleri

English: Increment and decrement operators, ++ and --, respectively, can be applied to numeric variables and have a high order of precedence compared to binary operators. In other words, they are often applied first in an expression.

Türkçe: Increment ve decrement operator'ları (sırasıyla ++ ve --) numeric variable'lara uygulanır ve binary operator'lara göre yüksek precedence'a sahiptir. Bu nedenle bir expression'da çoğunlukla önce uygulanırlar.

English: Increment and decrement operators require special care because the order in which they are attached to their associated variable can make a difference in how an expression is processed. Table 2.3 lists each of these operators.

Türkçe: Arttırma ve azaltma operatörleri özel dikkat gerektirir çünkü bunların ilişkili variable'a eklenme sırası, bir ifadenin işlenme biçiminde fark yaratabilir. Tablo 2.3 bu operatörlerin her birini listelemektedir.

English table caption: TABLE 2.3 Increment and decrement operators

Türkçe tablo başlığı: TABLO 2.3 Arttırma ve azaltma operatörleri

Operator	Example / Örnek	Description / Açıklama
Pre-increment	<code>++w</code>	Increases the value by 1 and returns the new value / Değeri 1 artırır ve yeni değeri döndürür
Pre-decrement	<code>--x</code>	Decreases the value by 1 and returns the new value / Değeri 1 azaltır ve yeni değeri döndürür
Post-increment	<code>y++</code>	Increases the value by 1 and returns the original value / Değeri 1 artırır ve eski değeri döndürür
Post-decrement	<code>z--</code>	Decreases the value by 1 and returns the original value / Değeri 1 azaltır ve eski değeri döndürür

English: The following code snippet illustrates this distinction:

Türkçe: Aşağıdaki code snippet bu ayrımı gösterir:

```
int parkAttendance = 0;
System.out.println(parkAttendance); // 0
System.out.println(++parkAttendance); // 1
```

Kaynak PDF sayfası 72

```
System.out.println(parkAttendance); // 1
System.out.println(parkAttendance--); // 1
System.out.println(parkAttendance); // 0
```

English: The first pre-increment operator updates the value for parkAttendance and outputs the new value of 1. The next post-decrement operator also updates the value of parkAttendance but outputs the value before the decrement occurs.

Türkçe: İlk pre-increment operator `parkAttendance` value'sunu günceller ve yeni value olan 1'i yazdırır. Sonraki post-decrement operator da `parkAttendance` value'sunu günceller; ancak decrement uygulanmadan önceki value'yu yazdırır.

English: For the exam, it is critical that you know the difference between expressions like `parkAttendance++` and `++parkAttendance`. The increment and decrement operators will be in multiple questions, and confusion about which value is returned could cause you to lose a lot of points on the exam.

Türkçe: Sınav için `parkAttendance++` ile `++parkAttendance` arasındaki farkı bilmek kritiktir. Increment ve decrement operator'ları birçok soruda görülebilir; expression'ın hangi value'yu ürettiğini karıştırmak hatalı yanıtı yol açar.

Working with Binary Arithmetic Operators

Türkçe başlık: Binary Arithmetic Operator'larla Çalışmak

English: Next, we move on to operators that take two operands, called binary operators. Binary operators are by far the most common operators in the Java language. They can be used to perform mathematical operations on variables, create logical expressions, and perform basic variable assignments. Binary operators are often combined in complex expressions with other binary operators; therefore, operator precedence is very important in evaluating expressions containing binary operators. In this section, we start with binary arithmetic operators; we expand to other binary operators in later sections.

Türkçe: Sırada iki operand alan binary operator'lar vardır. Bunlar Java'daki en yaygın operator türüdür; variable'lar üzerinde mathematical operation yapmak, logical expression oluşturmak ve temel assignment'ları gerçekleştirmek için kullanılır. Binary operator'lar karmaşık expression'larda sıkça birlikte kullanıldığından operator precedence önemlidir. Önce binary arithmetic operator'ları, ilerleyen bölümlerde diğer binary operator'ları ele alacağız.

Arithmetic Operators

Türkçe başlık: Aritmetik Operatörler

English: Arithmetic operators are those that operate on numeric values. They are shown in Table 2.4.

Türkçe: Aritmetik operator'lar sayısal değerler üzerinde işlem yapan operatörlerdir. Bunlar Tablo 2.4'te gösterilmektedir.

English table caption: TABLE 2.4 Binary arithmetic operators

Türkçe tablo başlığı: TABLO 2.4 İkili aritmetik operator'lar

Operator	Example / Örnek	Description / Açıklama
Addition	<code>a + b</code>	Adds two numeric values / İki numeric value'yu toplar
Subtraction	<code>c - d</code>	Subtracts two numeric values / Bir numeric value'dan diğerini çıkarır
Multiplication	<code>e * f</code>	Multiplies two numeric values / İki numeric value'yu çarpar
Division	<code>g / h</code>	Divides one numeric value by another / Bir numeric value'yu diğerine böler

Operator	Example / Örnek	Description / Açıklama
Modulus	<code>i % j</code>	Returns the remainder after division / Division sonrasındaki remainder'ı döndürür

Kaynak PDF sayfası 73

English: You should know all but modulus from early mathematics. If you don't know what modulus is, though, don't worry—we'll cover that shortly. Arithmetic operators also include the unary operators, ++ and --, which we covered already. As you may have noticed in Table 2.1, the multiplicative operators (*, /, %) have a higher order of precedence than the additive operators (+, -). Take a look at the following expression:

Türkçe: Modulus dışındaki işlemler temel matematikten tanındıktır; modulus birazdan açıklanacaktır. Arithmetic operator'lar daha önce ele alınan ++ ve -- gibi unary operator'ları da kapsar. Tablo 2.1'de görüldüğü üzere multiplicative operator'lar (*, /, %), additive operator'lardan (+, -) daha yüksek precedence'a sahiptir. Şu expression'ı inceleyin:

```
int price = 2 * 5 + 3 * 4 - 8;
```

English: First, you evaluate the `2 * 5` and `3 * 4`, which reduces the expression to this:

Türkçe: Öncelikle `2 * 5` ve `3 * 4` işlemlerini hesaplarsınız; bu, expression'ı şuna indirger:

```
int price = 10 + 12 - 8;
```

English: Then, you evaluate the remaining terms in left-to-right order, resulting in a value of price of 14. Make sure you understand why the result is 14 because you will likely see this kind of operator precedence question on the exam.

Türkçe: Ardından kalan terimleri soldan sağa evaluate edersiniz ve `price` value'su 14 olur. Sonucun neden 14 olduğunu anladığınızdan emin olun; sınavda bu tür operator precedence sorularıyla karşılaşmanız olasıdır.

English: All of the arithmetic operators may be applied to any Java primitives, with the exception of boolean. Furthermore, only the addition operators + and += may be applied to String values, which results in String concatenation. You will learn more about these operators and how they apply to String values in Chapter 4, "Core APIs."

Türkçe: Bütün arithmetic operator'lar, `boolean` dışındaki Java primitive'lerine uygulanabilir. Ayrıca `String` değerlerine yalnızca + ve += addition operator'ları uygulanabilir; bunun sonucu String concatenation'dır. Bu operator'ları ve `String` değerleriyle kullanımlarını Chapter 4 "Core APIs" içinde daha ayrıntılı öğreneceksiniz.

Adding Parentheses

Türkçe başlık: Parantez Ekleme

English: You might have noticed we said “Unless overridden with parentheses” prior to presenting Table 2.1 on operator precedence. That’s because you can change the order of operation explicitly by wrapping parentheses around the sections you want evaluated first.

Türkçe: Tablo 2.1'den önce “Parantezlerle değiştirilmedikçe” dediğimize dikkat edin. Öncelikli olarak hesaplanmasını istediğiniz bölümleri paranteze alarak gruplamayı açıkça değiştirebilirsiniz.

Changing the Order of Operation

Türkçe başlık: İşlem Sırasını Değiştirme

English: Let’s return to the previous price example. The following code snippet contains the same values and operators, in the same order, but with two sets of parentheses added:

Türkçe: Önceki `price` örneğine dönelim. Aşağıdaki code snippet aynı value ve operator’ları aynı sırada içerir; yalnızca iki parentheses grubu eklenmiştir:

```
int price = 2 * ((5 + 3) * 4 - 8);
```

English: This time you would evaluate the addition operator `5 + 3`, which reduces the expression to the following:

Türkçe: Bu kez `5 + 3` addition operation’ını değerlendirirsiniz; bu, expression’ı aşağıdaki hâle indirir:

```
int price = 2 * (8 * 4 - 8);
```

English: You can further reduce this expression by multiplying the first two values within the parentheses:

Türkçe: Parantez içindeki ilk iki value’yu çarparak expression’ı daha da sadeleştirebilirsiniz:

```
int price = 2 * (32 - 8);
```

English: Next, you subtract the values within the parentheses before applying terms outside the parentheses:

Türkçe: Daha sonra, parantezlerin dışındaki terimleri uygulamadan önce parantez içindeki değerleri çıkarırsınız:

```
int price = 2 * 24;
```

English: Finally, you would multiply the result by 2, resulting in a value of 48 for price.

Türkçe: Son olarak sonucu 2 ile çarparak fiyat için 48 değerini elde edersiniz.

Kaynak PDF sayfası 74

English: Parentheses can appear in nearly any question on the exam involving numeric values, so make sure you understand how they are changing the order of operation when you see them.

Türkçe: Sınavda sayısal değerler içeren hemen hemen her soruda parantezler görünebilir; bu nedenle, bunları gördüğünüzde işlem sırasını nasıl değiştirdiklerini anladığınızdan emin olun.

English: When you encounter code in your professional career in which you are not sure about the order of operation, feel free to add optional parentheses. While often not required, they can improve readability, especially as you'll see with ternary operators.

Türkçe: Mesleki kariyerinizde işleme sırasından emin olmadığınız bir kodla karşılaştığınızda isteğe bağlı parantez eklemekten çekinmeyin. Çoğu zaman gerekli olmasa da, özellikle üçlü operatörlerde göreceğiniz gibi okunabilirliği artırabilirler.

Verifying Parentheses Syntax

Türkçe başlık: Parantez Söz Dizimini Doğrulama

English: When working with parentheses, you need to make sure they are always valid and balanced.

Türkçe: Parantezlerle çalışırken bunların her zaman geçerli ve dengeli olduğundan emin olmanız gerekir.

English: Consider the following examples:

Türkçe: Aşağıdaki örnekleri göz önünde bulundurun:

```
long pigeon = 1 + ((3 * 5) / 3; // DOES NOT COMPILE
int blueJay = (9 + 2) + 3) / (2 * 4; // DOES NOT COMPILE
```

English: The first example does not compile because the parentheses are not balanced. There is a left parenthesis with no matching right parenthesis. The second example has an equal number of left and right parentheses, but they are not balanced properly. When reading from left to right, a new right parenthesis must match a previous left parenthesis. Likewise, all left parentheses must be closed by right parentheses before the end of the expression.

Türkçe: İlk örnek parantezlerin dengeli olmaması nedeniyle derlenmiyor. Eşleşen sağ parantez olmayan bir sol parantez var. İkinci örnekte eşit sayıda sol ve sağ parantez vardır ancak bunlar düzgün şekilde dengelenmemiştir. Soldan sağa okurken yeni sağ parantez önceki sol parantezle eşleşmelidir. Aynı şekilde ifadenin sonundan önce tüm sol parantezlerin sağ parantezlerle kapatılması gerekir.

English: Let's try another example:

Türkçe: Başka bir örnek deneyelim:

```
short robin = 3 + [(4 * 2) + 4]; // DOES NOT COMPILE
```

English: This example does not compile because Java, unlike some other programming languages, does not allow brackets, [], to be used in place of parentheses. If you replace the brackets with parentheses, the last example will compile just fine.

Türkçe: Bu örnek derlenmez; çünkü Java, bazı diğer dillerin aksine grouping parentheses () yerine square brackets [] kullanılmasına izin vermez. Köşeli parantezleri normal parantezlerle değiştirirseniz örnek derlenir.

Division and Modulus Operators

Türkçe başlık: Division ve Remainder Operator'ları

English: As we said earlier, the modulus operator, %, may be new to you. The modulus operator, sometimes called the remainder operator, is simply the remainder when two numbers are divided. For example, 9 divided by 3 divides evenly and has no remainder; therefore, the result of 9 % 3 is 0. On the other hand, 11 divided by 3 does not divide evenly; therefore, the result of 11 % 3 is 2.

Türkçe: Modulus operator (%) sizin için yeni olabilir. Remainder operator olarak da anılan %, iki sayının division işleminden kalanı üretir. Örneğin 9 % 3 sonucu 0, 11 % 3 sonucu 2'dir.

English: The following examples illustrate this distinction:

Türkçe: Aşağıdaki örnekler bu ayrımı göstermektedir:

```
System.out.println(9 / 3); // 3
System.out.println(9 % 3); // 0
System.out.println(10 / 3); // 3
System.out.println(10 % 3); // 1
System.out.println(11 / 3); // 3
System.out.println(11 % 3); // 2
```

Kaynak PDF sayfası 75

```
System.out.println(12 / 3); // 4
System.out.println(12 % 3); // 0
```

English: As you can see, the division results increase only when the value on the left side goes from 11 to 12, whereas the modulus remainder value increases by 1 each time the left side is increased until it wraps around to zero. For a given divisor y, the modulus operation results in a value between 0 and (y - 1) for positive dividends, or 0, 1, 2 in this example.

Türkçe: Görüldüğü gibi division sonucu yalnızca soldaki value 11'den 12'ye çıktığında artar. Modulus remainder ise soldaki value her artırıldığında bir artar ve sonra yeniden sıfıra döner. Belirli bir y divisor'ı için pozitif dividend'lerde modulus sonucu 0 ile y - 1 arasındadır; bu örnekte olası sonuçlar 0, 1 ve 2'dir.

English: Be sure to understand the difference between arithmetic division and modulus. For integer values, division results in the floor value of the nearest integer that fulfills the operation, whereas modulus is the remainder value. If you hear the phrase floor value, it just means the value without anything after the decimal point. For example, the floor value is 4 for each of the values 4.0, 4.5, and 4.9999999. Unlike rounding, which we'll cover in Chapter 4, you just take the value before the decimal point, regardless of what is after the decimal point.

Türkçe: Arithmetic division ile modulus arasındaki farkı iyi anlayın. Integer division'da ondalık kısım atılır; Java sonucu sıfıra doğru truncate eder. Modulus ise remainder value'yu verir. Kaynaktaki pozitif örneklerde 4.0, 4.5 ve 4.9999999 değerlerinin integer kısmı 4'tür. Bu işlem rounding değildir; decimal point'ten sonraki bölüm sonuca katılmaz.

English: The modulus operation is not limited to positive integer values in Java; it may also be applied to negative integers and floating-point numbers. For example, if the divisor is 5, then the modulus value of a negative number is between -4 and 0. For the exam, though, you are not required to be able to take the modulus of a negative integer or a floating-point number.

Türkçe: Java'da modül işlemi pozitif tamsayı değerleriyle sınırlı değildir; negatif tam sayılara ve kayan noktalı sayılara da uygulanabilir. Örneğin, bölen 5 ise negatif bir sayının modül değeri -4 ile 0 arasındadır. Ancak sınav için negatif bir tam sayının veya kayan noktalı sayının modülünü almanıza gerek yoktur.

Numeric Promotion

Türkçe başlık: Numeric Promotion

English: Now that you understand the basics of arithmetic operators, it is vital to talk about primitive numeric promotion, as Java may do things that seem unusual to you at first. As we showed in Chapter 1, "Building Blocks," each primitive numeric type has a bit-length. You don't need to know the exact size of these types for the exam, but you should know which are bigger than others. For example, you should know that a long takes up more space than an int, which in turn takes up more space than a short, and so on.

Türkçe: Artık aritmetik operatörlerin temellerini anladığınıza göre, primitive numeric promotion hakkında konuşmak çok önemlidir, çünkü Java ilk başta size alışılmadık görünen şeyler yapabilir. Bölüm 1, "Yapı Taşları"nda gösterdiğimiz gibi, her temel sayısal türün bir bit uzunluğu vardır. Sınav için bu türlerin tam boyutunu bilmenize gerek yok ancak hangilerinin diğerlerinden daha büyük olduğunu bilmelisiniz. Örneğin, long'un int'den daha fazla yer kapladığını, int'in de short'tan daha fazla yer kapladığını vb. bilmelisiniz.

English: You need to memorize certain rules that Java will follow when applying operators to data types:

Türkçe: Java operator'ları data type'lara uygularken izlenen şu kuralları öğrenmelisiniz:

Numeric Promotion Rules

Türkçe başlık: Numeric Promotion Kuralları

English: 1. If two values have different data types, Java will automatically promote one of the values to the larger of the two data types.

Türkçe: 1. İki value farklı data type'lardaysa Java küçük type'taki value'yu otomatik olarak büyük type'a promote eder.

English: 2. If one of the values is integral and the other is floating-point, Java will automatically promote the integral value to the floating-point value's data type.

Türkçe: 2. Value'lardan biri integral, diğeri floating-point ise Java integral value'yu floating-point value'nun data type'ına promote eder.

English: 3. Smaller data types, namely, byte, short, and char, are first promoted to int any time they're used with a Java binary arithmetic operator with a variable (as opposed to a value), even if neither of the operands is int.

Türkçe: 3. `byte`, `short` ve `char` gibi küçük data type'lar, literal yerine variable olarak binary arithmetic operator ile kullanıldıklarında iki operand da `int` olmasa bile önce `int`'e promote edilir.

English: 4. After all promotion has occurred and the operands have the same data type, the resulting value will have the same data type as its promoted operands.

Türkçe: 4. Bütün promotion'lar tamamlanıp operand'lar aynı data type'a geldikten sonra result, promote edilmiş operand'larla aynı data type'ta olur.

Kaynak PDF sayfası 76

English: The last two rules are the ones most people have trouble with and the ones likely to trip you up on the exam. For the third rule, note that unary operators are excluded from this rule. For example, applying ++ to a short value results in a short value.

Türkçe: Son iki kural, çoğu kişinin zorlandığı ve sınavda sizi yanıltabilecek kurallardır. Üçüncü kuralın unary operator'ları kapsamadığına dikkat edin. Örneğin `short` bir value'ya ++ uygulandığında result yine `short` type'ındadır.

Editör notu · Promotion'ın sınırı: Literal ve constant expression'lar da numeric promotion kurallarına tabidir. `byte b = 1 + 2;` örneğinde toplamın type'ı `int`'tir; `byte`a atamayı sağlayan ayrı constant narrowing kuralıdır. Unary +, - ve ~ de `byte/short/char` operand'ı `int`'e yükseltir; ++/-- variable'in type'ını korur. [JLS 17 §5.6](#).

English: Let's tackle some examples for illustrative purposes:

Türkçe: Açıklama amacıyla bazı örnekleri ele alalım:

English: - What is the data type of `x * y`?

Türkçe: - `x * y` expression'ının data type'ı nedir?

```
int x = 1;
long y = 33;
var z = x * y;
```

English: In this case, we follow the first rule. Since one of the values is int and the other is long, and long is larger than int, the int value x is first promoted to a long. The result z is then a long value.

Türkçe: Bu durumda ilk kural uygulanır. Value'lardan biri `int`, diğeri `long` olduğundan ve `long`, `int`ten büyük olduğundan `x` önce `long`a promote edilir. Sonuç olan `z` de `long` bir value'dur.

English: - What is the data type of `x + y`?

Türkçe: - `x + y` expression'ının data type'ı nedir?

```
double x = 39.21;
float y = 2.1;
var z = x + y;
```

English: This is actually a trick question, as the second line does not compile! As you may remember from Chapter 1, floating-point literals are assumed to be double unless postfixed with an f, as in 2.1f. If the value of y was set properly to 2.1f, then the promotion would be similar to the previous example, with both operands being promoted to a double, and the result z would be a double value.

Türkçe: İkinci satır derlenmediği için bu bir tuzak sorudur. Bölüm 1'den hatırlayacağınız üzere floating-point literal'lar, `2.1f` örneğindeki gibi `f` suffix'i taşımadıkça `double` kabul edilir. `y` value'su doğru biçimde `2.1f` yapılsaydı önceki örneğe benzer promotion uygulanırdı: iki operand da `double`a promote edilir ve `z` result'ı `double` olurdu.

English: - What is the data type of `x * y`?

Türkçe: - `x * y` expression'ının data type'ı nedir?

```
short x = 10;
short y = 3;
var z = x * y;
```

English: On the last line, we must apply the third rule: that `x` and `y` will both be promoted to `int` before the binary multiplication operation, resulting in an output of type `int`. If you were to try to assign the value to a `short` variable `z` without casting, then the code would not compile. Pay close attention to the fact that the resulting output is not a `short`, as we'll come back to this example in the upcoming "Assigning Values" section.

Türkçe: Son satırda üçüncü kural uygulanır: `x` ve `y`, binary multiplication'dan önce `int`te promote edilir; result da `int` type'ındadır. Bu value casting olmadan `short z` variable'ına atanırsa kod derlenmez. Sonucun `short` olmadığını özellikle unutmayın; bu örneğe sıradaki "Assigning Values" bölümünde döneceğiz.

English: - What is the data type of `w * x / y`?

Türkçe: - `w * x / y` expression'ının data type'ı nedir?

```
short w = 14;
float x = 13;
double y = 30;
var z = w * x / y;
```

English: In this case, we must apply all of the rules. First, `w` will automatically be promoted to `int` solely because it is a `short` and is being used in an arithmetic binary operation. The promoted `w` value will then be automatically promoted to a `float` so that it can be multiplied with `x`. The result of `w * x` will then be automatically promoted to a `double` so that it can be divided by `y`, resulting in a `double` value.

Türkçe: Bu örnekte bütün kurallar uygulanır. `w`, `short` olduğu ve binary arithmetic operation'da kullanıldığı için önce `int`te promote edilir. Ardından `x` ile multiplication için `w` value'su `float`a promote edilir. `w * x` result'ı da `y` ile division öncesinde `double`a promote edilir; final result `double` olur.

Kaynak PDF sayfası 77

English: When working with arithmetic operators in Java, you should always be aware of the data type of variables, intermediate values, and resulting values. You should apply operator precedence and parentheses and work outward, promoting data types along the way. In the next section, we'll discuss the intricacies of assigning these values to variables of a particular type.

Türkçe: Java arithmetic operator'larıyla çalışırken variable'ların, intermediate value'ların ve result'ların data type'larını sürekli izleyin. Operator precedence ile parentheses'i uygulayıp içten dışa ilerlerken gerekli numeric promotion'ları belirleyin. Sıradaki bölümde bu value'ların belirli type'lardaki variable'lara assignment ayrıntıları ele alınacaktır.

Assigning Values

Türkçe başlık: Değer Atama

English: Compilation errors from assignment operators are often overlooked on the exam, in part because of how subtle these errors can be. To be successful with the assignment operators, you should be fluent in understanding how the compiler handles numeric promotion and when casting is required. Being able to spot these issues is critical to passing the exam, as assignment operators appear in nearly every question with a code snippet.

Türkçe: Atama operatörlerinden kaynaklanan derleme hataları, kısmen bu hataların ne kadar incelikli olabileceğinden dolayı, genellikle sınavda gözden kaçırılır. Atama operatörlerinde başarılı olmak için, compiler'ın sayısal yükseltmeyi nasıl ele aldığını ve ne zaman casting yapılması gerektiğini anlama konusunda akıcı olmalısınız. Atama operatörleri neredeyse her soruda bir kod parçacığıyla birlikte görüldüğünden, bu sorunları tespit edebilmek sınavı geçmek için kritik öneme sahiptir.

Assignment Operator

Türkçe başlık: Atama Operatörü

English: An assignment operator is a binary operator that modifies, or assigns, the variable on the left side of the operator with the result of the value on the right side of the equation. Unlike most other Java operators, the assignment operator is evaluated from right to left.

Türkçe: Assignment operator, sağındaki expression'ın sonucunu soldaki variable'a atayan binary bir operatördür. Çoğu Java operator'ından farklı olarak sağdan sola birleşir: `a = b = 3`, `a = (b = 3)` biçiminde gruplanır.

English: The simplest assignment operator is the = assignment, which you have seen already:

Türkçe: En basit atama operator'ı, daha önce gördüğünüz = atamasıdır:

```
int herd = 1;
```

English: This statement assigns the herd variable the value of 1.

Türkçe: Bu statement `herd` variable'ına `1` value'sunu atar.

English: Java will automatically promote from smaller to larger data types, as you saw in the previous section on arithmetic operators, but it will throw a compiler exception if it detects that you are trying to convert from larger to smaller data types without casting. Table 2.5 lists the first assignment operator that you need to know for the exam. We present additional assignment operators later in this section.

Türkçe: Önceki arithmetic operator bölümünde gördüğünüz gibi Java küçük data type'lardan büyük data type'lara otomatik promotion uygular. Ancak büyük type'tan küçük type'a casting olmadan dönüşüm denendiğini saptarsa compiler error verir. Tablo 2.5 sınav için bilinmesi gereken ilk assignment operator'ı gösterir; ek assignment operator'lar bu bölümün ilerleyen kısmında ele alınır.

English table caption: TABLE 2.5 Simple assignment operator

Türkçe tablo başlığı: TABLO 2.5 Basit atama operator'ı

Operator	Example / Örnek	Description / Açıklama
Assignment	<code>int a = 50;</code>	Assigns the value on the right to the variable on the left / Sağdaki value'yu soldaki variable'a atar

Casting Values

Türkçe başlık: Değerlere Cast Uygulama

English: Seems easy so far, right? Well, we can't really talk about the assignment operator in detail until we've covered casting. Casting is a unary operation where one data type is explicitly interpreted as another data type. Casting is optional and unnecessary when converting to a

Türkçe: Buraya kadar kolay görünüyor, değil mi? Assignment operator'ı casting konusunu ele almadan ayrıntılı inceleyemeyiz. Casting, bir data type'ın açıkça başka bir data type olarak yorumlandığı unary operation'dır. Daha büyük bir type'a conversion yapılırken casting optional ve gereksizdir.

Kaynak PDF sayfası 78

English: larger or widening data type, but it is required when converting to a smaller or narrowing data type. Without casting, the compiler will generate an error when trying to put a larger data type inside a smaller one.

Türkçe: Buna karşılık daha küçük, yani narrowing bir data type'a conversion yapılırken casting zorunludur. Casting olmadan compiler, daha büyük bir data type'ı daha küçük bir data type'a yerleştirmeye çalışıldığında error üretir.

English: Casting is performed by placing the data type, enclosed in parentheses, to the left of the value you want to cast. Here are some examples of casting:

Türkçe: Cast uygulamak için hedef type parantez içinde değer soluna yazılır. Şu örnekleri inceleyin:

```
int fur = (int)5;
int hair = (short) 2;
String type = (String) "Bird";
short tail = (short)(4 + 10);
long feathers = 10(long); // DOES NOT COMPILE
```

English: Spaces between the cast and the value are optional. As shown in the second-to-last example, it is common for the right side to also be in parentheses. Since casting is a unary operation, it would only be applied to the 4 if we didn't enclose 4 + 10 in parentheses. The last example does not compile because the type is on the wrong side of the value.

Türkçe: Cast ile value arasındaki whitespace optional'dır. Sondan bir önceki örnekte olduğu gibi sağ tarafı parentheses içine almak yaygındır. Casting unary operation olduğundan 4 + 10 parentheses içine alınmasaydı cast yalnızca 4'e uygulanırdı. Son örnek, type value'nun yanlış tarafında bulunduğu için derlenmez.

English: On the one hand, it is convenient that the compiler automatically casts smaller data types to larger ones. On the other hand, it makes for great exam questions when they do the opposite to see whether you are paying attention. See if you can figure out why none of the following lines of code compile:

Türkçe: Bir yandan compiler'ın daha küçük veri türlerini daha büyük veri türlerine otomatik olarak dönüştürmesi uygundur. Öte yandan, dikkat edip etmediğinizi görmek için tam tersini yaptıklarında harika sınav soruları ortaya çıkar. Aşağıdaki kod satırlarından hiçbirinin neden derlenmediğini çözebilecek misiniz bir bakın:

```
float egg = 2.0 / 9; // DOES NOT COMPILE
int tadpole = (int)5 * 2L; // DOES NOT COMPILE
short frog = 3 - 2.0; // DOES NOT COMPILE
```

English: All of these examples involve putting a larger value into a smaller data type. Don't worry if you don't follow this quite yet; we cover more examples like this shortly.

Türkçe: Bu örneklerin tümü, daha büyük bir değeri daha küçük bir veri türüne yerleştirmeyi içerir. Bunu henüz tam olarak takip etmediyseniz endişelenmeyin; Yakında bunun gibi daha fazla örneği ele alacağız.

English: In this chapter, casting is primarily concerned with converting numeric data types into other data types. As you will see in later chapters, casting can also be applied to objects and references. In those cases, though, no conversion is performed. Put simply, casting a numeric value may change the data type, while casting an object only changes the reference to the object, not the object itself.

Türkçe: Bu bölümde casting öncelikle numeric data type'ların başka data type'lara dönüştürülmesiyle ilgilidir. İlerleyen bölümlerde casting'in object ve reference'lara da uygulanabildiğini göreceksiniz; ancak bu durumda object'in kendisi convert edilmez. Kısaca, numeric value'yu cast etmek data type'ı değiştirebilirken object'i cast etmek object'i değil, ona yönelik reference'ın type'ını değiştirir.

Reviewing Primitive Assignments

Türkçe başlık: Primitive Atamaları Gözden Geçirme

English: See if you can figure out why each of the following lines does not compile:

Türkçe: Aşağıdaki satırların her birinin neden derlenmediğini çözebilecek misiniz bir bakın:

```
int fish = 1.0; // DOES NOT COMPILE
short bird = 1921222; // DOES NOT COMPILE
int mammal = 9f; // DOES NOT COMPILE
long reptile = 192_301_398_193_810_323; // DOES NOT COMPILE
```

English: The first statement does not compile because you are trying to assign a double 1.0 to an integer value. Even though the value is a mathematic integer, by adding .0, you're instructing the compiler to treat it as a double. The second statement does not compile because the literal value 1921222 is outside the range of short, and the compiler detects this. The third statement does not compile because the f added to the end of the number

Türkçe: İlk statement, integer bir value'ya **double** literal **1.0** atamaya çalıştığınız için derlenmez. Value matematiksel olarak bir integer olsa da **.0** eklemek compiler'a onu **double** olarak değerlendirmesini söyler. İkinci statement derlenmez; çünkü **1921222** literal'ı **short** aralığının dışındadır ve compiler bunu belirler. Üçüncü statement derlenmez; çünkü sayının sonuna eklenen **f**

Kaynak PDF sayfası 79

English: instructs the compiler to treat the number as a floating-point value, but the assignment is to an int. Finally, the last statement does not compile because Java interprets the literal as an int and notices that the value is larger than int allows. The literal would need a postfix L or l to be considered a long.

Türkçe: compiler'a sayıyı floating-point bir value olarak değerlendirmesini söyler; ancak assignment bir **int**e yapılmaktadır. Son statement ise Java literal'ı **int** olarak yorumladığı ve value'nun **int** aralığını aştığını belirlediği için derlenmez. Literal'ın **long** sayılması için **L** veya **l** suffix'i gerekir.

Applying Casting

Türkçe başlık: Cast Uygulama

English: We can fix three of the previous examples by casting the results to a smaller data type.

Türkçe: Result'ları daha küçük bir data type'a **cast** ederek önceki örneklerden üçünü düzeltebiliriz.

English: Remember, casting primitives is required any time you are going from a larger numerical data type to a smaller numerical data type, or converting from a floating-point number to an integral value.

Türkçe: Daha büyük bir numeric data type'tan daha küçüğüne geçerken veya floating-point bir number'ı integral value'ya dönüştürürken primitive **casting** gerektiğini unutmayın.

```
int fish = (int)1.0;
short bird = (short)1921222; // Stored as 20678
int mammal = (int)9f;
```

English: What about applying casting to the last example?

Türkçe: Son örneğe casting uygulamaya ne dersiniz?

```
long reptile = (long)192301398193810323; // DOES NOT COMPILE
```

English: This still does not compile because the value is first interpreted as an int by the compiler and is out of range. The following fixes this code without requiring casting:

Türkçe: Değer ilk önce compiler tarafından int olarak yorumlandığından ve aralığın dışında olduğundan bu yine de derlenmiyor. Aşağıdakiler, bu kodu casting gerektirmeden düzeltir:

```
long reptile = 192301398193810323L;
```

Overflow and Underflow

Türkçe başlık: Overflow ve Underflow (Taşma ve Alt Taşma)

English: The expressions in the previous example now compile, although there's a cost. The second value, 1,921,222, is too large to be stored as a short, so numeric overflow occurs, and it becomes 20,678. Overflow is when a number is so large that it will no longer fit within the data type, so the system "wraps around" to the lowest negative value and counts up from there, similar to how modulus arithmetic works. There's also an analogous underflow, when the number is too low to fit in the data type, such as storing -200 in a byte field.

Türkçe: Önceki örnekteki statement'lar artık derlenir; ancak bunun bir bedeli vardır. İkinci value olan 1_921_222, short içinde saklanamayacak kadar büyük olduğundan numeric overflow oluşur ve sonuç 20_678 olur. Overflow, sayının data type'a sığmayacak kadar büyük olması ve sistemin en düşük negatif değere sarıp oradan saymaya devam etmesidir. Benzer biçimde, byte içinde -200 saklamaya çalışmak gibi değerin type'a sığmayacak kadar küçük olduğu durumda underflow oluşur.

English: This is beyond the scope of the exam but something to be careful of in your own code. For example, the following statement outputs a negative number:

Türkçe: Bu, sınavın kapsamı dışındadır ancak kendi kodunuzda dikkatli olmanız gereken bir şeydir. Örneğin, aşağıdaki ifade negatif bir sayı verir:

```
System.out.print(2147483647+1); // -2147483648
```

English: Since 2147483647 is the maximum int value, adding any strictly positive value to it will cause it to wrap to the smallest negative number.

Türkçe: 2147483647 maksimum int değeri olduğundan, buna kesinlikle pozitif bir değer eklemek, onun en küçük negatif sayıya sarılmasına neden olur.

English: Let's return to a similar example from the "Numeric Promotion" section earlier in the chapter.

Türkçe: Bu bölümün başındaki "Sayısal Yükseltme" kısmındaki benzer bir örneğe dönelim.

Kaynak PDF sayfası 80

```
short mouse = 10;
short hamster = 3;
short capybara = mouse * hamster; // DOES NOT COMPILE
```

English: Based on everything you have learned up until now about numeric promotion and casting, do you understand why the last line of this statement will not compile? As you may remember, short values are automatically promoted to int when applying any arithmetic operator, with the resulting value being of type int. Trying to assign a short variable with an int value results in a compiler error, as Java thinks you are trying to implicitly convert from a larger data type to a smaller one.

Türkçe: Numeric promotion ve casting hakkında şimdiye kadar öğrendiklerinize dayanarak son satırın neden derlenmediğini görebiliyor musunuz? Herhangi bir arithmetic operator uygulandığında short value'lar otomatik olarak inte promote edilir ve result int type'ında olur. Bir int value'yu short variable'a atamaya çalışmak compiler error oluşturur; çünkü Java bunu daha büyük bir data type'tan daha küçük olana implicit conversion denemesi olarak görür.

English: We can fix this expression by casting, as there are times that you may want to override the compiler's default behavior. In this example, we know the result of 10 * 3 is 30, which can easily fit into a short variable, so we can apply casting to convert the result back to a short:

Türkçe: Bazen compiler'ın default davranışını geçersiz kılmak isteyebilirsiniz; bu expression'ı casting ile düzeltebiliriz. Bu örnekte 10 * 3 sonucunun 30 olduğunu ve bir short variable'a sığacağını biliyoruz. Bu nedenle sonucu yeniden shorta cast edebiliriz:

```
short mouse = 10;
short hamster = 3;
short capybara = (short) (mouse * hamster);
```

English: By casting a larger value into a smaller data type, you instruct the compiler to ignore its default behavior. In other words, you are telling the compiler that you have taken additional steps to prevent overflow or underflow. It is also possible that in your particular application and scenario, overflow or underflow would result in acceptable values.

Türkçe: Daha büyük bir value'yu daha küçük bir data type'a cast ederek compiler'a default davranışını geçersiz kılmasını söylersiniz. Başka bir deyişle overflow veya underflow olasılığına karşı gerekli önlemleri aldığınızı bildirirsiniz. Uygulamanıza ve senaryonuza bağlı olarak overflow veya underflow sonucunda oluşan değerlerin kabul edilebilir olması da mümkündür.

English: Last but not least, casting can appear anywhere in an expression, not just on the assignment. For example, let's take a look at a modified form of the previous example:

Türkçe: Son olarak casting, yalnızca assignment üzerinde değil, bir expression'ın herhangi bir yerinde bulunabilir. Önceki örneğin değiştirilmiş biçimine bakalım:

```
short mouse = 10;
short hamster = 3;
short capybara = (short)mouse * hamster; // DOES NOT COMPILE
```

English: So, what's happening on the last line? Well, remember when we said casting was a unary operation? That means the cast in the last line is applied to mouse, and mouse alone. After the cast is complete, both operands are promoted to int since they are used with the binary multiplication operator (*), making the result an int and causing a compiler error.

Türkçe: Son satırda ne olur? Casting'in unary operation olduğunu hatırlayın. Bu nedenle son satırdaki cast yalnızca `mouse` variable'ına uygulanır. Cast tamamlandıktan sonra iki operand, binary multiplication operator (*) ile kullanıldıkları için `int`te promote edilir. Result `int` olur ve bu da compiler error'a yol açar.

English: What if we changed the last line to the following?

Türkçe: Son satırı aşağıdaki şekilde değiştirsek ne olur?

```
short capybara = 1 + (short)(mouse * hamster); // DOES NOT COMPILE
```

English: In the example, casting is performed successfully, but the resulting value is automatically promoted to int because it is used with the binary arithmetic operator (+).

Türkçe: Örnekte, casting başarılı bir şekilde gerçekleştirilir, ancak elde edilen değer, ikili aritmetik operator'ı (+) ile birlikte kullanıldığı için otomatik olarak int'ye yükseltilir.

Casting Values vs. Variables

Türkçe başlık: Değerlere ve Variable'lara Cast Uygulama

English: Revisiting our third numeric promotional rule, the compiler doesn't require casting when working with literal values that fit into the data type. Consider these examples:

Türkçe: Üçüncü `numeric promotion` kuralımıza dönersek compiler, data type'a sığan `literal` value'larla çalışırken casting istemez. Şu örnekleri inceleyin:

```
byte hat = 1;
byte gloves = 7 * 10;
short scarf = 5;
short boots = 2 + 1;
```

Kaynak PDF sayfası 81

English: All of these statements compile without issue. On the other hand, neither of these statements compiles:

Türkçe: Bu ifadelerin tümü sorunsuz biçimde derlenir. Buna karşılık aşağıdaki iki ifade de derlenmez:

```
short boots = 2 + hat; // DOES NOT COMPILE
byte gloves = 7 * 100; // DOES NOT COMPILE
```

English: The first statement does not compile because `hat` is a variable, not a value, and both operands are automatically promoted to `int`. When working with values, the compiler had enough information to determine the writer's intent. When working with variables, though, there is ambiguity about how to proceed, so the compiler reports an error. The second expression does not compile because `700` triggers an overflow for `byte`, which has a maximum value of `127`.

Türkçe: İlk ifade derlenmez; çünkü `hat` bir value değil variable'dır ve iki operand da otomatik olarak `int`te promote edilir. Compiler, doğrudan value'larla çalışırken yazarın niyetini belirleyecek kadar bilgiye sahiptir. Variable'larla çalışırken ise nasıl ilerleyeceği belirsizdir; bu yüzden compiler hata bildirir. İkinci expression da derlenmez; çünkü `700`, maksimum değeri `127` olan `byte` için overflow oluşturur.

Compound Assignment Operators

Türkçe başlık: Compound Assignment Operator'ları

English: Besides the simple assignment operator (`=`), Java supports numerous compound assignment operators. For the exam, you should be familiar with the compound operators in Table 2.6.

Türkçe: Java, basit assignment operator'a (`=`) ek olarak çok sayıda compound assignment operator destekler. Sınav için Tablo 2.6'daki compound operator'ları bilmelisiniz.

English table caption: TABLE 2.6 Compound assignment operators

Türkçe tablo başlığı: TABLO 2.6 Compound assignment operator'ları

Operator / Operator	Example / Örnek	Description / Açıklama
Addition assignment	<code>a += 5</code>	Adds the value on the right to the variable on the left and assigns the sum to the variable. / Sağdaki değeri soldaki variable'a ekler ve toplamı variable'a atar.
Subtraction assignment	<code>b -= 0.2</code>	Subtracts the value on the right from the variable on the left and assigns the difference to the variable. / Sağdaki değeri soldaki variable'dan çıkarır ve farkı variable'a atar.
Multiplication assignment	<code>c *= 100</code>	Multiplies the value on the right with the variable on the left and assigns the product to the variable. / Sağdaki değerle soldaki variable'ı çarpar ve çarpımı variable'a atar.
Division assignment	<code>d /= 4</code>	Divides the variable on the left by the value on the right and assigns the quotient to the variable. / Soldaki variable'ı sağdaki değere böler ve bölümü variable'a atar.

English: Compound operators are really just glorified forms of the simple assignment operator, with a built-in arithmetic or logical operation that applies the left and right sides of the statement and stores the resulting value in the variable on the left side of the statement. For example, the following two statements after the declaration of camel and giraffe are equivalent when run independently:

Türkçe: Compound operator'lar aslında basit assignment operator'ın, statement'ın sol ve sağ tarafına uygulanan yerleşik bir arithmetic veya logical operation içeren gelişmiş biçimleridir; ortaya çıkan değer statement'ın solundaki variable'da saklanır. Örneğin camel ve giraffe declaration'larından sonraki iki statement bağımsız çalıştırıldığında eşdeğerdir:

```
int camel = 2, giraffe = 3;
camel = camel * giraffe; // Simple assignment operator
camel *= giraffe;        // Compound assignment operator
```

Kaynak PDF sayfası 82

English: The left side of the compound operator can be applied only to a variable that is already defined and cannot be used to declare a new variable. In this example, if camel were not already defined, the expression camel *= giraffe would not compile.

Türkçe: Compound operator'ın sol tarafı yalnızca daha önce tanımlanmış bir variable'a uygulanabilir; yeni bir variable declare etmek için kullanılamaz. Bu örnekte camel daha önce tanımlanmamış olsaydı camel *= giraffe expression'ı derlenmezdi.

English: Compound operators are useful for more than just shorthand— they can also save you from having to explicitly cast a value. For example, consider the following. Can you figure out why the last line does not compile?

Türkçe: Compound operator'lar yalnızca kısa yazım sağlamaz; explicit cast gereksinimini de ortadan kaldırabilir. Aşağıdaki örnekte son satırın neden derlenmediğini düşünün.

```
long goat = 10;
int sheep = 5;
sheep = sheep * goat; // DOES NOT COMPILE
```

English: From the previous section, you should be able to spot the problem in the last line. We are trying to assign a long value to an int variable. This last line could be fixed with an explicit cast to (int), but there's a better way using the compound assignment operator:

Türkçe: Önceki bölümdeki kurala göre son satırdaki sorun açıktır: long value'yu int variable'a assign etmeye çalışıyoruz. Satır explicit (int) cast ile düzeltilebilir; ancak compound assignment operator daha kısa bir çözüm sunar:

```
long goat = 10;
int sheep = 5;
sheep *= goat;
```

English: The compound operator will first cast `sheep` to a `long`, apply the multiplication of two `long` values, and then cast the result to an `int`. Unlike the previous example, in which the compiler reported an error, the compiler will automatically cast the resulting value to the data type of the value on the left side of the compound operator.

Türkçe: Compound operator önce `sheep`i `long`a cast eder, iki `long` value'nun multiplication'ını uygular ve ardından result'ı `int`e cast eder. Compiler'ın error bildirdiği önceki örneğin aksine, result otomatik olarak compound operator'ın solundaki value'nun data type'ına cast edilir.

Return Value of Assignment Operators

Türkçe başlık: Atama Operatörlerinin Dönüş Değeri

English: One final thing to know about assignment operators is that the result of an assignment is an expression in and of itself equal to the value of the assignment. For example, the following snippet of code is perfectly valid, if a little odd-looking:

Türkçe: Assignment'ın kendisi de atanan değeri üreten bir expression'dır. Bu nedenle aşağıdaki kod alışılmadık görünse de tamamen geçerlidir:

```
long wolf = 5;
long coyote = (wolf=3);
System.out.println(wolf); // 3
System.out.println(coyote); // 3
```

English: The key here is that `(wolf=3)` does two things. First, it sets the value of the variable `wolf` to be 3. Second, it returns a value of the assignment, which is also 3.

Türkçe: Temel nokta, `(wolf = 3)` expression'ının iki iş yapmasıdır: Önce `wolf` variable'ına 3 atar, sonra assignment'ın yine 3 olan value'sunu döndürür.

English: The exam creators are fond of inserting the assignment operator (`=`) in the middle of an expression and using the value of the assignment as part of a more complex expression. For example, don't be surprised if you see an if statement on the exam similar to the following:

Türkçe: Sınavı oluşturanlar atama operatörünü (`=`) bir ifadenin ortasına eklemeyi ve atamanın değerini daha karmaşık bir ifadenin parçası olarak kullanmayı severler. Örneğin sınavda aşağıdakine benzer bir if ifadesi görürseniz şaşırmayın:

```
boolean healthy = false;
if(healthy = true)
System.out.print("Good!");
```

English: While this may look like a test if `healthy` is true, it's actually assigning `healthy` a value of `true`. The result of the assignment is the value of the assignment, which is `true`,

Türkçe: Bu ifade `healthy` variable'ının `true` olup olmadığını test ediyor gibi görünse de aslında `healthy`ye `true` atar. Assignment expression'ın result'ı, atanan value olan `true`dur;

Kaynak PDF sayfası 83

English: resulting in this snippet printing Good!. We'll cover this in more detail in the upcoming "Equality Operators" section.

Türkçe: dolayısıyla snippet `Good!` yazdırır. Bu ayrıntı sıradaki "Equality Operators" bölümünde daha kapsamlı ele alınacaktır.

Comparing Values

Türkçe başlık: Değerleri Karşılaştırma

English: The last set of binary operators revolves around comparing values. They can be used to check if two values are the same, check if one numeric value is less than or greater than another, and perform Boolean arithmetic. Chances are, you have used many of the operators in this section in your development experience.

Türkçe: Son binary operator grubu value karşılaştırmalarıyla ilgilidir. İki value'nun aynı olup olmadığını, bir numeric value'nun diğerinden küçük ya da büyük olup olmadığını sınamak ve boolean arithmetic yapmak için kullanılır. Bu bölümdeki operator'ların çoğunu geliştirme deneyiminizde muhtemelen kullanmışsınızdır.

Equality Operators

Türkçe başlık: Eşitlik Operatörleri

English: Determining equality in Java can be a nontrivial endeavor as there's a semantic difference between "two objects are the same" and "two objects are equivalent." It is further complicated by the fact that for numeric and boolean primitives, there is no such distinction.

Türkçe: Java'da equality belirlemek basit görünse de "iki object aynıdır" ile "iki object equivalent'tır" arasında semantic bir fark vardır. Numeric ve boolean primitive'lerde böyle bir ayrım bulunmaması konuyu daha da incelikli hâle getirir.

English: Table 2.7 lists the equality operators. The equals operator (==) and not equals operator (!=) compare two operands and return a boolean value determining whether the expressions or values are equal or not equal, respectively.

Türkçe: Tablo 2.7 equality operator'larını listeler. Equals operator (==) ve not-equals operator (!=) iki operand'ı karşılaştırır; sırasıyla expression veya value'ların equal ya da not equal olduğunu belirten bir boolean value döndürür.

English table caption: TABLE 2.7 Equality operators

Türkçe tablo başlığı: TABLO 2.7 Eşitlik operatörleri

Operator	Example / Örnek	Apply to primitives	Apply to objects
Equality	<code>a == 10</code>	Returns <code>true</code> if the two values represent the same value / Aynı value ise <code>true</code>	Returns <code>true</code> if both values reference the same object / Aynı object'i reference ediyorsa <code>true</code>

Operator	Example / Örnek	Apply to primitives	Apply to objects
Inequality	<code>b != 3.14</code>	Returns <code>true</code> if the two values represent different values / Farklı value ise <code>true</code>	Returns <code>true</code> if both values do not reference the same object / Farklı object'leri reference ediyorsa <code>true</code>

English: The equality operator can be applied to numeric values, boolean values, and objects (including String and null). When applying the equality operator, you cannot mix these types. Each of the following results in a compiler error:

Türkçe: Equality operator numeric value'lara, `boolean` value'lara ve object'lere (`String` ve `null` dahil) uygulanabilir. Equality operator kullanılırken bu kategoriler birbirine karıştırılmaz. Aşağıdakilerin her biri compiler error ile sonuçlanır:

```
boolean monkey = true == 3; // DOES NOT COMPILE
boolean ape = false != "Grape"; // DOES NOT COMPILE
boolean gorilla = 10.2 == "Koko"; // DOES NOT COMPILE
```

English: Pay close attention to the data types when you see an equality operator on the exam. As mentioned in the previous section, the exam creators also have a habit of mixing assignment operators and equality operators.

Türkçe: Sınavda bir eşitlik operator'ı gördüğünüzde veri türlerine çok dikkat edin. Önceki bölümde bahsedildiği gibi, sınav yaratıcılarının atama operatörleri ile eşitlik operatörlerini karıştırma alışkanlığı da vardır.

```
boolean bear = false;
```

Kaynak PDF sayfası 84

```
boolean polar = (bear = true);
System.out.println(polar); // true
```

English: At first glance, you might think the output should be false, and if the expression were `(bear == true)`, then you would be correct. In this example, though, the expression is assigning the value of true to bear, and as you saw in the section on assignment operators, the assignment itself has the value of the assignment. Therefore, polar is also assigned a value of true, and the output is true.

Türkçe: İlk bakışta output'un `false` olması gerektiğini düşünebilirsiniz; expression `(bear == true)` olsaydı bu doğru olurdu. Ancak burada expression, `bear` variable'ına `true` atar. Assignment'ın kendisi de atanan value'yu ürettiğinden `polar` variable'ına da `true` atanır ve output `true` olur.

English: For object comparison, the equality operator is applied to the references to the objects, not the objects they point to. Two references are equal if and only if they point to the same object or both point to null. Let's take a look at some examples:

Türkçe: Object comparison'da equality operator, object'lerin kendilerine değil onları gösteren reference'lara uygulanır. İki reference yalnızca aynı object'e işaret ediyorsa veya ikisi de `null` ise eşittir. Örneklere bakalım:

```
var monday = new File("schedule.txt");
var tuesday = new File("schedule.txt");
var wednesday = tuesday;
System.out.println(monday == tuesday); // false
System.out.println(tuesday == wednesday); // true
```

English: Even though all of the variables point to the same file information, only two references, tuesday and wednesday, are equal in terms of == since they point to the same object.

Türkçe: Bütün variable'lar aynı file bilgisini temsil etse de yalnızca tuesday ile wednesday aynı object'e işaret eder; bu yüzden yalnızca bu iki reference == açısından eşittir.

English: Wait, what's the File class? In this example, as well as during the exam, you may be presented with class names that are unfamiliar, such as File.

Türkçe: Bir dakika, File class'ı nedir? Bu örnekte ve sınavda File gibi tanımadığınız class adlarıyla karşılaşabilirsiniz.

English: Many times you can answer questions about these classes without knowing the specific details of these classes. In the previous example, you should be able to answer questions that indicate monday and tuesday are two separate and distinct objects because the new keyword is used, even if you are not familiar with the data types of these objects.

Türkçe: Çoğu zaman bu class'ların ayrıntılarını bilmeden de soruyu yanıtlayabilirsiniz. Önceki örnekte object'lerin data type'larına aşina olmasanız bile new keyword'ünün kullanılması, monday ile tuesday'ın iki ayrı object olduğunu anlamanız için yeterlidir.

English: In some languages, comparing null with any other value is always false, although this is not the case in Java.

Türkçe: Bazı dillerde null ile başka herhangi bir value'nun karşılaştırılması her zaman false verir; Java'da ise bu kural geçerli değildir.

```
System.out.print(null == null); // true
```

English: In Chapter 4, we'll continue the discussion of object equality by introducing what it means for two different objects to be equivalent. We'll also cover String equality and show how this can be a nontrivial topic.

Türkçe: Bölüm 4'te iki farklı object'in equivalent olmasının ne anlama geldiğini açıklayarak object equality konusuna devam edeceğiz. String equality de ele alınacak ve bunun basit olmayan, dikkat gerektiren bir konu olduğu gösterilecektir.

Relational Operators

Türkçe başlık: İlişkisel Operatörler

English: We now move on to relational operators, which compare two expressions and return a boolean value. Table 2.8 describes the relational operators you need to know for the exam.

Türkçe: Sırada iki expression'ı karşılaştırıp boolean value döndüren relational operator'lar vardır. Tablo 2.8 sınav için bilmeniz gereken relational operator'ları açıklar.

Kaynak PDF sayfası 85

English table caption: TABLE 2.8 Relational operators

Türkçe tablo başlığı: TABLO 2.8 İlişkisel operator'lar

Operator	Example / Örnek	Description / Açıklama
Less than	<code>a < 5</code>	Returns true if the value on the left is strictly less than the value on the right / Soldaki value kesin olarak daha küçükse <code>true</code> döndürür
Less than or equal to	<code>b <= 6</code>	Returns true if the value on the left is less than or equal to the value on the right / Soldaki value daha küçük veya eşitse <code>true</code> döndürür
Greater than	<code>c > 9</code>	Returns true if the value on the left is strictly greater than the value on the right / Soldaki value kesin olarak daha büyükse <code>true</code> döndürür
Greater than or equal to	<code>3 >= d</code>	Returns true if the value on the left is greater than or equal to the value on the right / Soldaki value daha büyük veya eşitse <code>true</code> döndürür
Type comparison	<code>e instanceof String</code>	Returns true if the reference on the left side is an instance of the type on the right side (class, interface, record, enum, annotation) / Soldaki reference sağdaki type'ın instance'ıysa <code>true</code> döndürür

Numeric Comparison Operators

Türkçe başlık: Sayısal Karşılaştırma Operatörleri

English: The first four relational operators in Table 2.8 apply only to numeric values. If the two numeric operands are not of the same data type, the smaller one is promoted, as previously discussed.

Türkçe: Tablo 2.8'deki ilk dört ilişkisel operator yalnızca sayısal değerlere uygulanır. İki sayısal operand aynı veri türünde değilse, daha önce tartışıldığı gibi küçük olan yükseltilir.

English: Let's look at examples of these operators in action:

Türkçe: Bu operatörlerin uygulamalı örneklerine bakalım:

```
int gibbonNumFeet = 2, wolfNumFeet = 4, ostrichNumFeet = 2;
System.out.println(gibbonNumFeet < wolfNumFeet); // true
System.out.println(gibbonNumFeet <= wolfNumFeet); // true
System.out.println(gibbonNumFeet >= ostrichNumFeet); // true
System.out.println(gibbonNumFeet > ostrichNumFeet); // false
```

English: Notice that the last example outputs false, because although gibbonNumFeet and ostrichNumFeet have the same value, gibbonNumFeet is not strictly greater than ostrichNumFeet.

Türkçe: Son örnek `false` yazdırır; çünkü `gibbonNumFeet` ile `ostrichNumFeet` aynı value'ya sahip olsa da `gibbonNumFeet`, `ostrichNumFeet`ten kesin olarak büyük değildir.

instanceof Operator

Türkçe başlık: `instanceof` Operator'ı

English: The final relational operator you need to know for the exam is the instanceof operator, shown in Table 2.8. It is useful for determining whether an arbitrary object is a member of a particular class or interface at runtime.

Türkçe: Sınav için bilmeniz gereken son relational operator, Tablo 2.8'de gösterilen `instanceof` operator'ıdır. Runtime'da herhangi bir object'in belirli bir class veya interface'in instance'ı olup olmadığını belirlemek için kullanılır.

English: Why wouldn't you know what class or interface an object is? As we will get into in Chapter 6, "Class Design," Java supports polymorphism. For now, all you need to know is

Türkçe: Bir object'in runtime'daki class veya interface type'ını neden önceden bilmiyor olabiliriz? Bölüm 6 "Class Design"da ele alınacağı üzere Java polymorphism'i destekler. Şimdilik bilmeniz gereken şudur:

Kaynak PDF sayfası 86

English: objects can be passed around using a variety of references. For example, all classes inherit from `java.lang.Object`. This means that any instance can be assigned to an `Object` reference. For example, how many objects are created and used in the following code snippet?

Türkçe: Object'ler çeşitli reference'lar aracılığıyla aktarılabilir. Örneğin bütün class'lar `java.lang.Object`ten kalıtım alır; bu nedenle herhangi bir instance bir `Object` reference'a atanabilir. Aşağıdaki code snippet'te kaç object oluşturulup kullanılmaktadır?

```
Integer zooTime = Integer.valueOf(9);
Number num = zooTime;
Object obj = zooTime;
```

English: In this example, only one object is created in memory, but there are three different references to it because Integer inherits both Number and Object. This means that you can call instanceof on any of these references with three different data types, and it will return true for each of them.

Türkçe: Bu örnekte bellekte yalnızca bir object oluşturulur; ancak `Integer`, hem `Number` hem de `Object` class'larından kalıtım aldığı için bu object'e yönelik üç farklı reference vardır. Dolayısıyla bu reference'ların her birinde `instanceof` üç farklı data type ile kullanılabilir ve her durumda `true` döner.

English: Where polymorphism often comes into play is when you create a method that takes a data type with many possible subclasses. For example, imagine that we have a function that opens the zoo and prints the time. As input, it takes a Number as an input parameter.

Türkçe: Polymorphism çoğunlukla, birçok olası subclass'ı bulunan bir data type'ı parameter olarak alan method'larda kullanılır. Örneğin hayvanat bahçesini açıp saati yazdıran ve input parameter olarak `Number` alan bir method düşünün.

```
public void openZoo(Number time) {}
```

English: Now, we want the function to add O'clock to the end of output if the value is a whole number type, such as an Integer; otherwise, it just prints the value.

Türkçe: Value `Integer` gibi bir whole-number type ise method'un output sonuna `O'clock` eklemesini, aksi durumda yalnızca value'yu yazdırmasını istiyoruz.

```
public void openZoo(Number time) {  
    if (time instanceof Integer)  
        System.out.print((Integer)time + " O'clock");  
    else  
        System.out.print(time);  
}
```

English: We now have a method that can intelligently handle both Integer and other values. A good exercise left for the reader is to add checks for other numeric data types such as Short, Long, Double, and so on.

Türkçe: Artık hem `Integer` hem de diğer değerleri uygun biçimde işleyebilen bir method'umuz var. Okuyucu için iyi bir alıştırmaya, `Short`, `Long`, `Double` gibi diğer numeric type'lar için kontroller eklemektir.

English: Notice that we cast the Integer value in this example. It is common to use casting with instanceof when working with objects that can be various different types, since casting gives you access to fields available only in the more specific classes. It is considered a good coding practice to use the instanceof operator prior to casting from one object to a narrower type.

Türkçe: Bu örnekte `Integer` değerini cast ettiğimize dikkat edin. Farklı type'larda olabilen object'lerle çalışırken casting ile `instanceof` birlikte sık kullanılır; çünkü casting, yalnızca daha specific class'larda bulunan field'lara erişim sağlar. Bir object'i daha dar bir type'a cast etmeden önce `instanceof` kullanmak iyi bir kodlama pratiğidir.

English: For the exam, you only need to focus on when instanceof is used with classes and interfaces. Although it can be used with other high-level types, such as records, enums, and annotations, it is not common.

Türkçe: Sınav için `instanceof` operator'ının class ve interface'lerle ne zaman kullanıldığına odaklanmanız yeterlidir. Record, enum ve annotation gibi diğer üst düzey type'larla da kullanılabilir; ancak bu yaygın değildir.

Invalid instanceof

Türkçe başlık: Geçersiz `instanceof`

English: One area the exam might try to trip you up on is using instanceof with incompatible types. For example, Number cannot possibly hold a String value, so the following causes a compilation error:

Türkçe: Sınavın yanıtlanabileceği noktalardan biri, `instanceof` operator'ını incompatible type'larla kullanmaktır. Örneğin `Number` hiçbir durumda `String` value tutamaz; bu yüzden aşağıdaki kod compilation error oluşturur:

```
public void openZoo(Number time) {  
    if(time instanceof String) // DOES NOT COMPILE  
        System.out.print(time);  
}
```

Kaynak PDF sayfası 87

English: If the compiler can determine that a variable cannot possibly be cast to a specific class, it reports an error.

Türkçe: Compiler bir variable'ın belirli bir class'a cast edilmesinin mümkün olmadığını saptarsa hata bildirir.

null and the instanceof operator

Türkçe başlık: `null` ve `instanceof` Operator'ı

English: What happens if you call instanceof on a null variable? For the exam, you should know that calling instanceof on the null literal or a null reference always returns false.

Türkçe: `null` bir variable üzerinde `instanceof` kullanırsanız ne olur? Sınav için, `null` literal'ı veya `null` reference ile `instanceof` kullanımının her zaman `false` döndürdüğünü bilmelisiniz.

```
System.out.print(null instanceof Object); // false  
Object noObjectHere = null;  
System.out.print(noObjectHere instanceof String); // false
```

English: The preceding examples both print false. It almost doesn't matter what the right side of the expression is. We say "almost" because there are exceptions. This example does not compile, since null is used on the right side of the instanceof operator:

Türkçe: Önceki iki örnek de `false` yazdırır. Expression'ın sağ tarafında ne olduğu neredeyse önemsizdir. "Neredeyse" diyoruz; çünkü istisnalar vardır. Aşağıdaki örnek, `instanceof` operator'ının sağ tarafında `null` kullanıldığı için derlenmez:

```
System.out.print(null instanceof null); // DOES NOT COMPILE
```

English: Although it may feel like you've learned everything there is about the instanceof operator, there's a lot more coming! In Chapter 3, we introduce pattern matching with the instanceof operator, which was officially added in Java 16. In Chapter 7, "Beyond Classes," we introduce polymorphism in much more detail and show how to apply these rules to interfaces.

Türkçe: `instanceof` operator'ı hakkında her şeyi öğrenmiş gibi hissedebilirsiniz; ancak ileride daha fazlası var. Chapter 3'te Java 16'da resmen eklenen `instanceof` pattern matching tanıtılır. Chapter 7 "Beyond Classes" içinde ise polymorphism daha ayrıntılı ele alınır ve bu kuralların interface'lere nasıl uygulandığı gösterilir.

Logical Operators

Türkçe başlık: Mantıksal Operatörler

English: If you have studied computer science, you may have already come across logical operators before. If not, no need to panic— we'll be covering them in detail in this section.

Türkçe: Bilgisayar bilimi okuduysanız daha önce mantıksal operatörlerle karşılaşmış olabilirsiniz. Değilse paniğe gerek yok; bu bölümde bunları ayrıntılı olarak ele alacağız.

English: The logical operators, (&), (|), and (^), may be applied to both numeric and boolean data types; they are listed in Table 2.9. When they're applied to boolean data types, they're referred to as logical operators. Alternatively, when they're applied to numeric data types, they're referred to as bitwise operators, as they perform bitwise comparisons of the bits that compose the number. For the exam, though, you don't need to know anything about numeric bitwise comparisons, so we'll leave that educational aspect to other books.

Türkçe: `&`, `|` ve `^` operator'ları hem numeric hem de boolean data type'lara uygulanabilir; Tablo 2.9'da listelenirler. Boolean data type'lara uygulandıklarında logical operator, numeric data type'lara uygulandıklarında ise sayıyı oluşturan bitleri karşılaştırdıkları için bitwise operator adını alırlar. Sınavda numeric bitwise comparison ayrıntılarını bilmeniz gerekmez.

English table caption: TABLE 2.9 Logical operators

Türkçe tablo başlığı: TABLO 2.9 Mantıksal operator'lar

Operator	Example / Örnek	Description / Açıklama
Logical AND	<code>a & b</code>	<code>true</code> only if both values are <code>true</code> / Yalnızca iki value da <code>true</code> ise <code>true</code>
Logical inclusive OR	<code>c \ d</code>	<code>true</code> if at least one value is <code>true</code> / En az bir value <code>true</code> ise <code>true</code>
Logical exclusive OR	<code>e ^ f</code>	<code>true</code> only if exactly one value is <code>true</code> / Yalnızca bir value <code>true</code> ise <code>true</code>

Kaynak PDF sayfası 88

English: You should familiarize yourself with the truth tables in Figure 2.2, where x and y are assumed to be boolean data types.

Türkçe: Şekil 2.2'deki x ve y'nin boolean veri türleri olduğu kabul edilen doğruluk tablolarına kendinizi alıştırmalısınız.

English figure caption: FIGURE 2.2 The logical truth tables for `&`, `|`, and `^`

Türkçe şekil başlığı: ŞEKİL 2.2 `&`, `|` ve `^` için mantıksal doğruluk tabloları

Operator / Operator	x	y	Result / Sonuç
AND (<code>x & y</code>)	<code>true</code>	<code>true</code>	<code>true</code>
AND (<code>x & y</code>)	<code>true</code>	<code>false</code>	<code>false</code>
AND (<code>x & y</code>)	<code>false</code>	<code>true</code>	<code>false</code>
AND (<code>x & y</code>)	<code>false</code>	<code>false</code>	<code>false</code>
INCLUSIVE OR (<code>x \ y</code>)	<code>true</code>	<code>true</code>	<code>true</code>
INCLUSIVE OR (<code>x \ y</code>)	<code>true</code>	<code>false</code>	<code>true</code>
INCLUSIVE OR (<code>x \ y</code>)	<code>false</code>	<code>true</code>	<code>true</code>
INCLUSIVE OR (<code>x \ y</code>)	<code>false</code>	<code>false</code>	<code>false</code>
EXCLUSIVE OR (<code>x ^ y</code>)	<code>true</code>	<code>true</code>	<code>false</code>
EXCLUSIVE OR (<code>x ^ y</code>)	<code>true</code>	<code>false</code>	<code>true</code>
EXCLUSIVE OR (<code>x ^ y</code>)	<code>false</code>	<code>true</code>	<code>true</code>
EXCLUSIVE OR (<code>x ^ y</code>)	<code>false</code>	<code>false</code>	<code>false</code>

English: Here are some tips to help you remember this table:

Türkçe: Bu tabloyu hatırlamanıza yardımcı olacak bazı ipuçları:

English: - AND is only true if both operands are true.

Türkçe: - AND yalnızca iki operand da `true` ise `true`dur.

English: - Inclusive OR is only false if both operands are false.

Türkçe: - Inclusive OR yalnızca iki operand da `false` ise `false`dur.

English: - Exclusive OR is only true if the operands are different.

Türkçe: - Exclusive OR yalnızca operand'lar farklıysa `true`dur.

English: Let's take a look at some examples:

Türkçe: Bazı örneklere göz atalım:

```
boolean eyesClosed = true;
boolean breathingSlowly = true;
boolean resting = eyesClosed | breathingSlowly;
boolean asleep = eyesClosed & breathingSlowly;
boolean awake = eyesClosed ^ breathingSlowly;
System.out.println(resting); // true
System.out.println(asleep); // true
System.out.println(awake); // false
```

English: You should try these out yourself, changing the values of `eyesClosed` and `breathingSlowly` and studying the results.

Türkçe: `eyesClosed` ve `breathingSlowly` value'larını değiştirip sonuçları inceleyerek bu ifadeleri kendiniz denemelisiniz.

Conditional Operators

Türkçe başlık: Koşullu Operatörler

English: Next, we present the conditional operators, `&&` and `||`, in Table 2.10.

Türkçe: Daha sonra, `&&` ve `||` koşullu operatörlerini Tablo 2.10'da sunuyoruz.

Kaynak PDF sayfası 89

English table caption: TABLE 2.10 Conditional operators

Türkçe tablo başlığı: TABLO 2.10 Koşullu operator'lar

Operator	Example / Örnek	Description / Açıklama
Conditional AND	<code>a && b</code>	<code>true</code> only if both values are <code>true</code> ; if the left side is <code>false</code> , the right side is not evaluated / Yalnızca iki value da <code>true</code> ise <code>true</code> ; sol taraf <code>false</code> ise sağ taraf değerlendirilmez

Operator	Example / Örnek	Description / Açıklama
Conditional OR	<code>c d</code>	<code>true</code> if at least one value is <code>true</code> ; if the left side is <code>true</code> , the right side is not evaluated / En az bir value <code>true</code> ise <code>true</code> ; sol taraf <code>true</code> ise sağ taraf değerlendirilmez

English: The conditional operators, often called short-circuit operators, are nearly identical to the logical operators, & and |, except that the right side of the expression may never be evaluated if the final result can be determined by the left side of the expression. For example, consider the following statement:

Türkçe: Genellikle kısa devre operatörleri olarak adlandırılan koşullu operator'lar, & ve | mantıksal operatörleriyle hemen hemen aynıdır; ancak, nihai sonuç, ifadenin sol tarafı tarafından belirlenebiliyorsa, ifadenin sağ tarafı hiçbir zaman değerlendirilemeyebilir. Örneğin aşağıdaki ifadeyi göz önünde bulundurun:

```
int hour = 10;
boolean zooOpen = true || (hour < 4);
System.out.println(zooOpen); // true
```

English: Referring to the truth tables, the value zooOpen can be false only if both sides of the expression are false. Since we know the left side is true, there's no need to evaluate the right side, since no value of hour will ever make this code print false. In other words, hour could have been -10 or 892; the output would have been the same. Try it yourself with different values for hour!

Türkçe: Truth table'a göre zooOpen ancak expression'ın iki tarafı da false olduğunda false olabilir. Sol tarafın true olduğunu bildiğimiz için sağ taraf değerlendirilmez; hiçbir hour value'su output'u değiştiremez. hour -10 veya 892 olsaydı da output aynı kalırdı. Farklı hour value'larıyla deneyebilirsiniz.

Avoiding a NullPointerException

Türkçe başlık: NullPointerException'dan Kaçınmak

English: A more common example of where conditional operators are used is checking for null objects before performing an operation. In the following example, if duck is null, the program will throw a NullPointerException at runtime:

Türkçe: Conditional operator'ların yaygın kullanım alanlarından biri, operation öncesinde object'in null olup olmadığını kontrol etmektir. Aşağıdaki örnekte duck null ise program runtime'da NullPointerException fırlatabilir:

```
if(duck!=null & duck.getAge()<5) { // Could throw a NullPointerException
// Do something
}
```

English: The issue is that the logical AND (&) operator evaluates both sides of the expression. We could add a second if statement, but this could get unwieldy if we have a lot of variables to check. An easy-to-read solution is to use the conditional AND operator (&&):

Türkçe: Sorun, logical AND operator (&) expression'ın iki tarafını da değerlendirmesidir. İkinci bir `if` statement eklenebilir; ancak kontrol edilecek çok variable varsa çözüm karmaşıklaşır. Daha okunaklı çözüm conditional AND operator (&&) kullanmaktır:

```
if(duck!=null && duck.getAge()<5) {  
    // Do something  
}
```

English: In this example, if duck is null, the conditional prevents a NullPointerException from ever being thrown, since the evaluation of `duck.getAge() < 5` is never reached.

Türkçe: Bu örnekte `duck null` ise `duck.getAge() < 5` hiç değerlendirilmez; böylece condition `NullPointerException` fırlatılmasını önler.

Kaynak PDF sayfası 90

Checking for Unperformed Side Effects

Dil çalışması: `side effect` için [ünite sözlüğü](#); cümle yapıları için [grammar notu](#).

Türkçe başlık: Atlanan Side Effect'leri İzleme

English: Be wary of short-circuit behavior on the exam, as questions are known to alter a variable on the right side of the expression that may never be reached. This is referred to as an unperformed side effect. For example, what is the output of the following code?

Türkçe: Sınavda short-circuit davranışına dikkat edin: Sağ operand'daki bir variable değişikliği, o operand atlandığı için hiç gerçekleşmeyebilir. Kaynak bunu unperformed side effect olarak adlandırır. Örneğin aşağıdaki kodun çıktısı nedir?

```
int rabbit = 6;  
boolean bunny = (rabbit >= 6) || (++rabbit <= 7);  
System.out.println(rabbit);
```

English: Because `rabbit >= 6` is true, the increment operator on the right side of the expression is never evaluated, so the output is 6.

Türkçe: `rabbit >= 6` zaten `true` olduğundan expression'ın sağ tarafındaki increment operator hiç değerlendirilmez; dolayısıyla çıktı 6 olur.

Making Decisions with the Ternary Operator

Türkçe başlık: Üçlü Operatör

English: The final operator you should be familiar with for the exam is the conditional operator, `?:`, otherwise known as the ternary operator. It is notable in that it is the only operator that takes three operands. The ternary operator has the following form:

Türkçe: Sınav için bilmeniz gereken son operator, ternary operator olarak da adlandırılan conditional operator `?:` dir. Üç operand alan tek operator'dır ve şu formdadır:

```
booleanExpression ? expression1 : expression2
```

English: The first operand must be a boolean expression, and the second and third operands can be any expression that returns a value. The ternary operation is really a condensed form of a combined if and else statement that returns a value. We cover if/else statements in a lot more detail in Chapter 3, so for now we just use simple examples.

Türkçe: İlk operand bir boolean expression olmalıdır; ikinci ve üçüncü operand'lar ise value döndüren herhangi bir expression olabilir. Ternary operation aslında value döndüren birleşik `if/else` statement'ının kısaltılmış biçimidir. `if/else` statement'ları Chapter 3'te çok daha ayrıntılı ele alınacağından burada yalnızca basit örnekler kullanıyoruz.

English: For example, consider the following code snippet that calculates the food amount for an owl:

Türkçe: Örneğin, bir baykuşun yiyecek miktarını hesaplayan aşağıdaki kod parçasını düşünün:

```
int owl = 5;
int food;
if(owl < 2) {
    food = 3;
} else {
    food = 4;
}
System.out.println(food); // 4
```

English: Compare the previous code snippet with the following ternary operator code snippet:

Türkçe: Önceki kod pasajını aşağıdaki üçlü operator kod pasajıyla karşılaştırın:

```
int owl = 5;
int food = owl < 2 ? 3 : 4;
System.out.println(food); // 4
```

Kaynak PDF sayfası 91

English: These two code snippets are equivalent. Note that it is often helpful for readability to add parentheses around the expressions in ternary operations, although doing so is certainly not required. It is especially helpful when multiple ternary operators are used together, though.

Türkçe: Bu iki code snippet eşdeğerdir. Ternary operation'lardaki expression'ların çevresine parentheses eklemek okunabilirliği çoğunlukla artırır; ancak zorunlu değildir. Birden fazla ternary operator birlikte kullanıldığında parentheses eklemek özellikle yararlıdır.

English: Consider the following two equivalent expressions:

Türkçe: Aşağıdaki iki eşdeğer ifadeyi göz önünde bulundurun:

```
int food1 = owl < 4? owl > 2? 3: 4: 5;
int food2 = (owl < 4? ((owl > 2)? 3: 4): 5);
```

English: While they are equivalent, we find the second statement far more readable. That said, it is possible the exam could use multiple ternary operators in a single line.

Türkçe: Bunlar eşdeğer olsa da ikinci ifadeyi çok daha okunaklı buluyoruz. Bununla birlikte, sınavın tek bir satırda birden fazla üçlü operator'ı kullanması mümkündür.

English: For the exam, you should know that there is no requirement that second and third expressions in ternary operations have the same data types, although it does come into play when combined with the assignment operator. Compare the two statements following the variable declaration:

Türkçe: Sınav için, ternary operation'daki ikinci ve üçüncü expression'ın aynı data type'ta olmasının zorunlu olmadığını bilin. Ancak expression assignment operator ile birlikte kullanıldığında type uyumluluğu önem kazanır. Variable declaration'dan sonraki iki statement'ı karşılaştırın:

OCP teknik notu: Seçilmeyen operand runtime'da çalışmaz; buna rağmen bütün expression derleme sırasında type kontrolünden geçer. Sonuç type'ını yalnız o çalıştırmada seçilen değere bakarak belirleme.

```
int stripes = 7;
System.out.print((stripes > 5)? 21: "Zebra");
int animal = (stripes < 9)? 3: "Horse"; // DOES NOT COMPILE
```

English: Both expressions evaluate similar boolean values and return an int and a String, although only the first one will compile. System.out.print() does not care that the expressions are completely different types, because it can convert both to Object values and call toString() on them. On the other hand, the compiler does know that "Horse" is of the wrong data type and cannot be assigned to an int; therefore, it does not allow the code to be compiled.

Türkçe: İki expression da benzer boolean koşulları değerlendirip bir `int` ve bir `String` döndürür; ancak yalnızca ilki derlenir. `System.out.print()` farklı type'lardaki expression'ları kabul eder; ikisini de `Object`ye dönüştürüp `toString()` çağırabilir. Buna karşılık compiler, "Horse"un `int` variable'a atanamayacak yanlış data type'ta olduğunu bilir ve ikinci kodun derlenmesine izin vermez.

Ternary Expression and Unperformed Side Effects

Türkçe başlık: Ternary Expression ve Atlanan Side Effect'ler

English: As we saw with the conditional operators, a ternary expression can contain an unperformed side effect, as only one of the expressions on the right side will be evaluated at runtime. Let's illustrate this principle with the following example:

Türkçe: Conditional operator'larda olduğu gibi ternary expression'da da bir side effect atlanabilir: Son iki operand'dan yalnızca seçilen operand runtime'da değerlendirilir. Şu örneği inceleyin:

```
int sheep = 1;
int zzz = 1;
int sleep = zzz<10? sheep++: zzz++;
System.out.print(sleep + "," + zzz); // 2,1
```

English: Notice that since the left-hand boolean expression was true, only sheep was incremented. Contrast the preceding example with the following modification:

Türkçe: Sol taraftaki boolean expression **true** olduğu için yalnızca **sheep**in increment edildiğine dikkat edin. Önceki örneği aşağıdaki değişikliklerle karşılaştırın:

Kaynak PDF sayfası 92

```
int sheep = 1;
int zzz = 1;
int sleep = sheep>=10? sheep++: zzz++;
System.out.print(sleep + "," + zzz); // 1,2
```

English: Now that the left-hand boolean expression evaluates to false, only zzz is incremented.

Türkçe: Sol taraftaki boolean expression artık **false** olarak değerlendirildiği için yalnızca **zzz** increment edilir.

English: In this manner, we see how the changes in a ternary operator may not be applied if the particular expression is not used.

Türkçe: Bu şekilde, belirli bir ifade kullanılmadığı takdirde üçlü bir operatördeki değişikliklerin nasıl uygulanamayacağını görüyoruz.

English: For the exam, be wary of any question that includes a ternary expression in which a variable is modified in one of the expressions on the right-hand side.

Türkçe: Sınavda, sağ taraftaki ifadelerden birinde bir variable'ın değiştirildiği üçlü bir ifade içeren herhangi bir soruya karşı dikkatli olun.

Summary

Türkçe başlık: Özet

English: This chapter covered a wide variety of Java operator topics for unary, binary, and ternary operators. Hopefully, most of these operators were review for you. If not, you need to study them in detail. It is important that you understand how to use all of the required Java operators covered in this chapter and know how operator precedence and parentheses influence the way a particular expression is interpreted.

Türkçe: Bu bölümde unary, binary ve ternary Java operator'ları ele alındı. Bunların çoğu sizin için tekrar niteliğinde değilse ayrıntılı çalışmanız gerekir. Bölümdeki operator'ların kullanımını ve operator precedence ile parentheses'in expression evaluation üzerindeki etkisini anlamak önemlidir.

English: There will likely be numerous questions on the exam that appear to test one thing, such as NIO.2 or exception handling, when in fact the answer is related to the misuse of a particular operator that causes the application to fail to compile. When you see an operator involving numbers on the exam, always check that the appropriate data types are used and that they match each other where applicable.

Türkçe: Sınavda NIO.2 veya exception handling konusunu ölçüyor gibi görüldüğü hâlde aslında yanlış operator kullanımı nedeniyle derlenmeyen birçok soru olabilir. Numeric value içeren bir operator gördüğünüzde uygun data type'ların kullanıldığını ve gerektiğinde birbiriyle compatible olduğunu kontrol edin.

English: Operators are used throughout the exam, in nearly every code sample, so the better you understand this chapter, the more prepared you will be for the exam.

Türkçe: Operator'lar sınavdaki neredeyse her code sample'da kullanılır; bu nedenle bölümü ne kadar iyi anlarsanız sınava o kadar hazırlıklı olursunuz.

Exam Essentials

Türkçe başlık: Sınav Esasları

English: Be able to write code that uses Java operators. This chapter covered a wide variety of operator symbols. Go back and review them several times so that you are familiar with them throughout the rest of the book.

Türkçe: Java operator'larını kullanan kod yazabilmelisiniz. Bu bölüm çok çeşitli operator sembollerini ele aldı. Kitabın geri kalanında rahatça tanıyabilmek için bunları birkaç kez gözden geçirin.

English: Be able to recognize which operators are associated with which data types. Some operators may be applied only to numeric primitives, some only to boolean values, and some only to objects. It is important that you notice when an operator and operand(s) are mismatched, as this issue is likely to come up in a couple of exam questions.

Türkçe: Hangi operator'ın hangi data type'la kullanılabildiğini tanıyabilmelisiniz. Bazı operator'lar yalnızca numeric primitive'lere, bazıları yalnızca boolean value'lara, bazıları ise object'lere uygulanabilir. Operator ile operand arasındaki type uyumsuzluğunu fark etmek önemlidir; bu tuzak sınav sorularında sık kullanılır.

Kaynak PDF sayfası 93

English: Understand when casting is required or numeric promotion occurs. Whenever you mix operands of two different data types, the compiler needs to decide how to handle the resulting data type. When you're converting from a smaller to a larger data type, numeric promotion is automatically applied. When you're converting from a larger to a smaller data type, casting is required.

Türkçe: Casting'in ne zaman gerektiğini ve numeric promotion'ın ne zaman gerçekleştiğini anlayın. Farklı data type'lardaki iki operand birlikte kullanıldığında compiler result'ın data type'ını belirler. Küçük type'tan büyük type'a dönüşümde numeric promotion otomatik uygulanır; büyük type'tan küçük type'a dönüşümde casting gerekir.

English: Understand Java operator precedence. Most Java operators you'll work with are binary, but the number of expressions is often greater than two. Therefore, you must understand the order in which Java will evaluate each operator symbol.

Türkçe: Java operator önceliğini anlayın. Çalışacağınız çoğu Java operator'ı ikili operatördür ancak ifadelerin sayısı genellikle ikiden fazladır. Bu nedenle, Java'nın her operator sembolünü değerlendireceği sırayı anlamalısınız.

English: Be able to write code that uses parentheses to override operator precedence. You can use parentheses in your code to manually change the order of precedence.

Türkçe: Operator precedence'ı geçersiz kılmak için parentheses kullanan kod yazabilmelisiniz. Precedence sırasını elle değiştirmek için kodunuzda parentheses kullanabilirsiniz.

Review Questions

Türkçe başlık: İnceleme Soruları

English: The answers to the chapter review questions can be found in the Appendix.

Türkçe: Bölüm inceleme sorularının yanıtlarını Ek'te bulabilirsiniz.

Question 1 / Soru 1

English: 1. Which of the following Java operators can be used with boolean variables? (Choose all that apply.)

Türkçe: 1. Aşağıdaki Java operator'larından hangileri `boolean` variable'larla kullanılabilir? (Uygun olanların tümünü seçin.)

- A. `==`
- B. `+`
- C. `--`
- D. `!`
- E. `%`
- F. `~`
- G. `Cast with (boolean)`

Türkçe seçenek notu: G. `(boolean)` ile casting

Question 2 / Soru 2

English: 2. What data type (or types) will allow the following code snippet to compile? (Choose all that apply.)

Türkçe: 2. Aşağıdaki code snippet'in derlenmesini hangi data type veya type'lar sağlar? (Uygun olanların tümünü seçin.)

```
byte apples = 5;  
short oranges = 10;  
_____ bananas = apples + oranges;
```

- A. `int`
- B. `long`
- C. `boolean`
- D. `double`
- E. `short`
- F. `byte`

Question 3 / Soru 3

English: 3. What change, when applied independently, would allow the following code snippet to compile? (Choose all that apply.)

Türkçe: 3. Aşağıdaki code snippet'e birbirinden bağımsız uygulandığında hangi değişiklikler kodun derlenmesini sağlar? (Uygun olanların tümünü seçin.)

```
3: long ear = 10;  
4: int hearing = 2 * ear;
```

English: A. No change; it compiles as is.

Türkçe: A. Değişiklik gerekmez; kod olduğu gibi derlenir.

English: B. Cast ear on line 4 to int.

Türkçe: B. 4. satırdaki `ear` variable'ını `int`te cast edin.

English: C. Change the data type of ear on line 3 to short.

Türkçe: C. 3. satırdaki `ear` variable'ının data type'ını `short` olarak değiştirin.

English: D. Cast `2 * ear` on line 4 to int.

Türkçe: D. 4. satırdaki `2 * ear` expression'ını `int`te cast edin.

English: E. Change the data type of hearing on line 4 to short.

Türkçe: E. 4. satırdaki `hearing` variable'ının data type'ını `short` olarak değiştirin.

English: F. Change the data type of hearing on line 4 to long.

Türkçe: F. 4. satırdaki `hearing` variable'ının data type'ını `long` olarak değiştirin.

Question 4 / Soru 4

English: 4. What is the output of the following code snippet?

Türkçe: 4. Aşağıdaki code snippet'in output'u nedir?

```
3: boolean canine = true, wolf = true;
4: int teeth = 20;
5: canine = (teeth != 10) ^ (wolf=false);
6: System.out.println(canine+", "+teeth+", "+wolf);
```

- A. true, 20, true
- B. true, 20, false
- C. false, 10, true
- D. false, 20, false
- E. The code will not compile because of line 5.
- F. None of the above.

Türkçe seçenekler: E. Kod 5. satır nedeniyle derlenmez. F. Yukarıdakilerin hiçbiri.

Question 5 / Soru 5

English: 5. Which of the following operators are ranked in increasing or the same order of precedence? Assume the + operator is binary addition, not the unary form. (Choose all that apply.)

Türkçe: 5. Aşağıdaki operator dizilerinden hangileri artan veya aynı precedence sırasındadır? + operator'ının unary değil binary addition biçimi olduğunu varsayın. (Uygun olanların tümünü seçin.)

- A. +, *, %, --
- B. ++, (int), *
- C. =, ==, !
- D. (short), =, !, *
- E. *, /, %, +, ==
- F. !, ||, &
- G. ^, +, =, +=

Question 6 / Soru 6

English: 6. What is the output of the following program?

Türkçe: 6. Aşağıdaki programın output'u nedir?

```
1: public class CandyCounter {
2:     static long addCandy(double fruit, float vegetables) {
3:         return (int)fruit+vegetables;
4:     }
5:
6:     public static void main(String[] args) {
7:         System.out.print(addCandy(1.4, 2.4f) + ", ");
8:         System.out.print(addCandy(1.9, (float)4) + ", ");
9:         System.out.print(addCandy((long)(int)(short)2, (float)4)); } }
```

- A. 4, 6, 6.0
- B. 3, 5, 6
- C. 3, 6, 6
- D. 4, 5, 6
- E. The code does not compile because of line 9.
- F. None of the above.

Türkçe seçenekler: E. Kod 9. satır nedeniyle derlenmez. F. Yukarıdakilerin hiçbiri.

Question 7 / Soru 7

English: 7. What is the output of the following code snippet?

Türkçe: 7. Aşağıdaki code snippet'in output'u nedir?

```
int ph = 7, vis = 2;
boolean clear = vis > 1 & (vis < 9 || ph < 2);
boolean safe = (vis > 2) && (ph++ > 1);
boolean tasty = 7 <= --ph;
System.out.println(clear + "-" + safe + "-" + tasty);
```

- A. true-true-true
- B. true-true-false
- C. true-false-true
- D. true-false-false
- E. false-true-true
- F. false-true-false
- G. false-false-true
- H. false-false-false

Question 8 / Soru 8

English: 8. What is the output of the following code snippet?

Türkçe: 8. Aşağıdaki code snippet'in output'u nedir?

```
4: int pig = (short)4;
5: pig = pig++;
6: long goat = (int)2;
7: goat -= 1.0;
8: System.out.print(pig + " - " + goat);
```

```
A. 4 - 1
B. 4 - 2
C. 5 - 1
D. 5 - 2
E. The code does not compile due to line 7.
F. None of the above.
```

Türkçe seçenekler: E. Kod 7. satır nedeniyle derlenmez. F. Yukarıdakilerin hiçbiri.

Question 9 / Soru 9

English: 9. What are the unique outputs of the following code snippet? (Choose all that apply.)

Türkçe: 9. Aşağıdaki code snippet'in benzersiz output'ları nelerdir? (Uygun olanların tümünü seçin.)

```
int a = 2, b = 4, c = 2;
System.out.println(a > 2 ? --c : b++);
System.out.println(b = (a!=c ? a : b++));
System.out.println(a > b ? b < c ? b : 2 : 1);
```

```
A. 1
B. 2
C. 3
D. 4
E. 5
F. 6
G. The code does not compile.
```

Türkçe seçenek notu: G. Kod derlenmez.

Question 10 / Soru 10

English: 10. What are the unique outputs of the following code snippet? (Choose all that apply.)

Türkçe: 10. Aşağıdaki code snippet'in benzersiz output'ları nelerdir? (Uygun olanların tümünü seçin.)

```
short height = 1, weight = 3;
short zebra = (byte) weight * (byte) height;
double ox = 1 + height * 2 + weight;
long giraffe = 1 + 9 % height + 1;
System.out.println(zebra);
System.out.println(ox);
System.out.println(giraffe);
```

```
A. 1
B. 2
C. 3
D. 4
E. 5
F. 6
G. The code does not compile.
```

Türkçe seçenek notu: G. Kod derlenmez.

Question 11 / Soru 11

English: 11. What is the output of the following code?

Türkçe: 11. Aşağıdaki kodun output'u nedir?

```
11: int sample1 = (2 * 4) % 3;  
12: int sample2 = 3 * 2 % 3;  
13: int sample3 = 5 * (1 % 2);  
14: System.out.println(sample1 + ", " + sample2 + ", " + sample3);
```

- A. 0, 0, 5
- B. 1, 2, 10
- C. 2, 1, 5
- D. 2, 0, 5
- E. 3, 1, 10
- F. 3, 2, 6
- G. The code does not compile.

Türkçe seçenek notu: G. Kod derlenmez.

Question 12 / Soru 12

English: 12. The _____ operator increases a value and returns the original value, while the _____ operator decreases a value and returns the new value.

Türkçe: 12. _____ operator'ı value'yu artırıp original value'yu döndürür; _____ operator'ı ise value'yu azaltıp new value'yu döndürür.

- A. post-increment, post-increment
- B. pre-decrement, post-decrement
- C. post-increment, post-decrement
- D. post-increment, pre-decrement
- E. pre-increment, pre-decrement
- F. pre-increment, post-decrement

Question 13 / Soru 13

English: 13. What is the output of the following code snippet?

Türkçe: 13. Aşağıdaki code snippet'in output'u nedir?

```
boolean sunny = true, raining = false, sunday = true;  
boolean goingToTheStore = sunny & raining ^ sunday;  
boolean goingToTheZoo = sunday && !raining;  
boolean stayingHome = !(goingToTheStore && goingToTheZoo);  
System.out.println(goingToTheStore + "-" + goingToTheZoo  
    + "-" + stayingHome);
```

- A. true-false-false
- B. false-true-false
- C. true-true-true
- D. false-true-true
- E. false-false-false
- F. true-true-false
- G. None of the above

Türkçe seçenek notu: G. Yukarıdakilerin hiçbiri.

Question 14 / Soru 14

English: 14. Which of the following statements are correct? (Choose all that apply.)

Türkçe: 14. Aşağıdaki statement'lerden hangileri doğrudur? (Uygun olanların tümünü seçin.)

English: A. The return value of an assignment operation expression can be void.

Türkçe: A. Assignment operation expression'ının return value'su `void` olabilir.

English: B. The inequality operator (!=) can be used to compare objects.

Türkçe: B. Inequality operator (!=) object'leri karşılaştırmak için kullanılabilir.

English: C. The equality operator (==) can be used to compare a boolean value with a numeric value.

Türkçe: C. Equality operator (==), boolean value ile numeric value'yu karşılaştırmak için kullanılabilir.

English: D. During runtime, the & and | operators may cause only the left side of the expression to be evaluated.

Türkçe: D. Çalışma zamanı sırasında & ve | operator'lar ifadenin yalnızca sol tarafının değerlendirilmesine neden olabilir.

English: E. The return value of an assignment operation expression is the value of the newly assigned variable.

Türkçe: E. Assignment operation expression'ının return value'su, variable'a yeni atanan value'dur.

English: F. In Java, 0 and false may be used interchangeably.

Türkçe: F. Java'da 0 ile `false` birbirinin yerine kullanılabilir.

English: G. The logical complement operator (!) cannot be used to flip numeric values.

Türkçe: G. Logical complement operator (!) numeric value'ları tersine çevirmek için kullanılamaz.

Question 15 / Soru 15

English: 15. Which operators take three operands or values? (Choose all that apply.)

Türkçe: 15. Hangi operator'lar üç operand veya value alır? (Uygun olanların tümünü seçin.)

- A. =
- B. &&
- C. *=
- D. ?:
- E. &
- F. ++
- G. /

Question 16 / Soru 16

English: 16. How many lines of the following code contain compiler errors?

Türkçe: 16. Aşağıdaki kodun kaç satırı compiler error içerir?

```
int note = 1 * 2 + (long)3;  
short melody = (byte)(double)(note *= 2);  
double song = melody;  
float symphony = (float)((song == 1_000f) ? song * 2L : song);
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4

Question 17 / Soru 17

English: 17. Given the following code snippet, what are the values of the variables after it is executed? (Choose all that apply.)

Türkçe: 17. Aşağıdaki code snippet çalıştırıldıktan sonra variable'ların value'ları nedir? (Uygun olanların tümünü seçin.)

```
int ticketsTaken = 1;  
int ticketsSold = 3;  
ticketsSold += 1 + ticketsTaken++;  
ticketsTaken *= 2;  
ticketsSold += (long)1;
```

English: A. ticketsSold is 8.

Türkçe: A. `ticketsSold` value'su 8dir.

English: B. ticketsTaken is 2.

Türkçe: B. `ticketsTaken` value'su 2'dir.

English: C. ticketsSold is 6.

Türkçe: C. `ticketsSold` value'su 6'dır.

English: D. ticketsTaken is 6.

Türkçe: D. `ticketsTaken` value'su 6'dır.

English: E. ticketsSold is 7.

Türkçe: E. `ticketsSold` value'su 7'dir.

English: F. ticketsTaken is 4.

Türkçe: F. `ticketsTaken` value'su 4'tür.

English: G. The code does not compile.

Türkçe: G. Kod derlenmez.

Question 18 / Soru 18

English: 18. Which of the following can be used to change the order of operation in an expression? (Choose all that apply.)

Türkçe: 18. Bir expression'daki operation order'ı değiştirmek için aşağıdakilerden hangileri kullanılabilir? (Uygun olanların tümünü seçin.)

- A. []
- B. < >
- C. ()
- D. \ /
- E. { }
- F. " "

Question 19 / Soru 19

English: 19. What is the result of executing the following code snippet? (Choose all that apply.)

Türkçe: 19. Aşağıdaki code snippet çalıştırıldığında sonuç nedir? (Uygun olanların tümünü seçin.)

```
3: int start = 7;
4: int end = 4;
5: end += ++start;
6: start = (byte) (Byte.MAX_VALUE + 1);
```

English: A. `start` is 0.

Türkçe: A. `start` değeri 0'dır.

English: B. `start` is -128.

Türkçe: B. `start` değeri -128'dir.

English: C. `start` is 127.

Türkçe: C. `start` değeri 127'dir.

English: D. `end` is 8.

Türkçe: D. `end` değeri 8'dir.

English: E. `end` is 11.

Türkçe: E. `end` değeri 11'dir.

English: F. `end` is 12.

Türkçe: F. `end` değeri 12'dir.

English: G. The code does not compile.

Türkçe: G. Kod derlenmez.

English: H. The code compiles but throws an exception at runtime.

Türkçe: H. Kod derlenir; ancak runtime'da exception fırlatır.

Question 20 / Soru 20

English: 20. Which of the following statements about unary operators are true? (Choose all that apply.)

Türkçe: 20. Unary operator'larla ilgili aşağıdaki statement'lardan hangileri doğrudur? (Uygun olanların tümünü seçin.)

English: A. Unary operators are always executed before any surrounding numeric binary or ternary operators.

Türkçe: A. Unary operator'lar her zaman çevrelerindeki numeric binary veya ternary operator'lardan önce yürütülür.

English: B. The `-` operator can be used to flip a boolean value.

Türkçe: B. `-` operator'ı boolean value'yu tersine çevirmek için kullanılabilir.

English: C. The pre-increment operator (`++`) returns the value of the variable before the increment is applied.

Türkçe: C. Pre-increment operator (`++`), increment uygulanmadan önce variable'ın değerini döndürür.

English: D. The post-decrement operator (`--`) returns the value of the variable before the decrement is applied.

Türkçe: D. Post-decrement operator (`--`), decrement uygulanmadan önce variable'ın değerini döndürür.

English: E. The `!` operator cannot be used on numeric values.

Türkçe: E. `!` operator'ı numeric value'larda kullanılamaz.

English: F. None of the above

Türkçe: F. Yukarıdakilerin hiçbiri.

Question 21 / Soru 21

English: 21. What is the result of executing the following code snippet?

Türkçe: 21. Aşağıdaki code snippet çalıştırıldığında sonuç nedir?

```
int myFavoriteNumber = 8;
int bird = ~myFavoriteNumber;
int plane = -myFavoriteNumber;
var superman = bird == plane ? 5 : 10;
System.out.println(bird + "," + plane + "," + --superman);
```

- A. -7,-8,9
- B. -7,-8,10
- C. -8,-8,4
- D. -8,-8,5
- E. -9,-8,9
- F. -9,-8,10
- G. None of the above

Türkçe seçenek notu: G. Yukarıdakilerin hiçbiri.

Kaynak dışı teknik pekiştirme

Bu belgedeki kaynak akışına ek açıklama karıştırılmamıştır. Java 17/OCP karar kartları, exam trap'ler ve active recall çalışmaları ayrı [technical memory notes](#) belgesinde korunur.

Kapsam doğrulaması

Kapsam özeti: 0065–0100 aralığındaki 36/36 kaynak sayfa doğrulandı; eksik sayfa yoktur.

Appendix · Kaynak cevaplarıyla kontrol

Bu bölüm, kaynak kitabın **Appendix: Answers to the Review Questions** bölümündeki Chapter 2 cevaplarından hazırlanmış özgün Türkçe çözüm rehberidir; İngilizce açıklamaların birebir çevirisi ve gerçek OCP sınav cevapları değildir. Kaynak: [ana PDF](#), fiziksel PDF sayfaları 913–916. **Official Answer** başlıkları kitabın kaynak cevaplarına karşılık gelir.

Önce soruyu kapalı notla çöz; seçtiğin her harfin yanına bir cümle gerekçe yaz. Sonra aşağıdan kontrol et. Yanlış seçenek veya yanlış gerekçe, hata günlüğüne ayrı kayıt olarak girer. Kaynakta tespit edilen anlatım sorunları **Editör notu** olarak ayrılmıştır.

Official Answer 1 / Kaynak Cevap 1

Kaynak cevap: A, D, G. [Soru 1](#)

`==`, `!` ve zaten boolean olan değere (`boolean`) cast uygulanabilir. Sayısal değeri boolean'a dönüştürme desteklenmez; aritmetik ve `~` boolean kabul etmez.

Official Answer 2 / Kaynak Cevap 2

Kaynak cevap: A, B, D. [Soru 2](#)

`byte + short` sonucu `int`tir; `int`, `long` ve `double` hedefler uygundur. `short/byte` hedefleri cast olmadan kabul edilmez; boolean sayısal tür değildir.

Official Answer 3 / Kaynak Cevap 3

Kaynak cevap: B, C, D, F. [Soru 3](#)

`2 * ear` ifadesi `long` olur. Uygun daraltma veya hedefi `long` yapma derlemeyi sağlar; daha küçük hedef tür seçmek sorunu çözmez.

Official Answer 4 / Kaynak Cevap 4

Kaynak cevap: B. [Soru 4](#)

Atama ifadesi (`wolf = false`) hem `wolf`u değiştirir hem `false` üretir. `true ^ false` sonucu `true` olduğundan çıktı `true`, `20`, `false` olur; `teeth` değişmez.

Official Answer 5 / Kaynak Cevap 5

Kaynak cevap: A, C. [Soru 5](#)

İstenen sıra düşük öncelikten yükseğe doğrudur; aynı düzey de kabul edilir. B ve E ters yöndedir; diğer diziler öncelik gruplarını karıştırır.

Official Answer 6 / Kaynak Cevap 6

Kaynak cevap: F. [Soru 6](#)

(`int`) yalnız `fruit` operand'ına uygulanır; `+ vegetables` sonucu tekrar `float` olur ve `long` dönüş türüne sığdırılmaz. Tüm toplamı cast etmek farklı bir programdır; verilen kod derlenmez.

Official Answer 7 / Kaynak Cevap 7

Kaynak cevap: D. [Soru 7](#)

Sonuç `true-false-false` olur. `&&` sol tarafı false olduğundan `ph++` çalışmaz; son `--ph` değeri 6 yapar ve `7 <= 6` false olur.

Official Answer 8 / Kaynak Cevap 8

Kaynak cevap: A. [Soru 8](#)

`pig = pig++` eski değeri tekrar atadığı için `pig` 4 kalır; `goat -= 1.0` örtük cast ile geçerlidir. Çıktı `4 - 1`; compound assignment'ı normal atamayla karıştırma.

Official Answer 9 / Kaynak Cevap 9

Kaynak cevap: A, D, E. [Soru 9](#)

Sırayla `4`, `5`, `1` yazdırılır. Ternary yalnız seçilen kolu çalıştırır; `b = b++` adımı artan değeri eski sonuçla tekrar değiştirilir.

Official Answer 10 / Kaynak Cevap 10

Kaynak cevap: G. [Soru 10](#)

İki operand'ı ayrı ayrı `byte` cast etmek çarpımın türünü `byte` yapmaz; binary numeric promotion sonucu `int`'tir. Bu sonucu `short`a doğrudan atayan satır yüzünden program derlenmez.

Official Answer 11 / Kaynak Cevap 11

Kaynak cevap: D. [Soru 11](#)

`*` ile `%` aynı öncelik düzeyindedir ve soldan birleşir; üç sonuç `2`, `0`, `5` olur. `%`, bölümün kalanını verir.

Official Answer 12 / Kaynak Cevap 12

Kaynak cevap: D. [Soru 12](#)

Post-increment eski değeri üretip değişkeni artırır; pre-decrement önce azaltıp yeni değeri üretir. “pre/post” ile “increment/decrement” iki ayrı ayrımdır.

Official Answer 13 / Kaynak Cevap 13

Kaynak cevap: F. [Soru 13](#)

Çıktı `true-true-false` olur: `(sunny & raining) ^ sunday` true, ikinci ifade true ve son olumsuzlama false'tur. **Editör notu:** Kitap açıklamasındaki aynı öncelik iddiası yanlıştır; Java 17'de `&`, `^`dan daha yüksek önceliklidir. Cevap harfi değişmez.

Official Answer 14 / Kaynak Cevap 14

Kaynak cevap: B, E, G. [Soru 14](#)

Reference'larda `==`/`!=` kullanılabilir, atama yeni değeri üretir ve sayısal işareti `-` değiştirir. Boolean ile sayı karşılaştırılmaz; `|` kısa devre yapmaz.

Official Answer 15 / Kaynak Cevap 15

Kaynak cevap: D. [Soru 15](#)

?: üç operand alır. ++ unary, kalan seçenekler binary olduğundan uzun bir ifadede kullanılsalar da üç operand'lı operatör sayılmazlar.

Official Answer 16 / Kaynak Cevap 16

Kaynak cevap: B. [Soru 16](#)

Yalnız ilk satır derlenmez: toplamdaki (long) 3, sonucu long yapar ama hedef inttir. Sonraki satırlardaki uygun cast/dönüşümler ayrı değerlendirilir.

Official Answer 17 / Kaynak Cevap 17

Kaynak cevap: C, F. [Soru 17](#)

Son değerler ticketsTaken = 4, ticketsSold = 6 olur. Postfix ilk toplamda eski 1'i kullanır; son += (long) 1 örtük cast sayesinde derlenir.

Official Answer 18 / Kaynak Cevap 18

Kaynak cevap: C. [Soru 18](#)

Gruplamayı değiştiren işaret () parantezidir. [], {} veya <> bu amaçla aritmetik parantez yerine geçmez.

Official Answer 19 / Kaynak Cevap 19

Kaynak cevap: B, F. [Soru 19](#)

Önce start 8, end 12 olur; ardından 128'i bytea daraltmak -128 üretir. Bu tamsayı dönüşümünde exception beklemek yanlıştır.

Official Answer 20 / Kaynak Cevap 20

Kaynak cevap: A, D, E. [Soru 20](#)

Kitabın beklediği cevap A, D, E'dir; postfix eski değeri verir ve ! sayısal değer kabul etmez. **Sınır notu:** A'daki "always" ifadesini parantezle değiştirilmemiş öncelik grubu için oku: -(1 + 2) içinde toplama önce tamamlanır; precedence, bütün çalışma sırasını tek başına açıklamaz.

Official Answer 21 / Kaynak Cevap 21

Kaynak cevap: E. [Soru 21](#)

~8 sonucu -9, -8 sonucu -8'dir; eşit olmadıkları için ternary 10'u seçer. --superman 9 üretir ve çıktı -9, -8, 9 olur.